



VELS

INSTITUTE OF SCIENCE, TECHNOLOGY & ADVANCED STUDIES (VISTAS)
(Deemed to be University Estd. u/s 3 of the UGC Act, 1956)
PALLAVARAM, THALAMBUR, PERIYAPALAYAM, THIRUVANMIYUR - CHENNAI





DEPARTMENT OF DATA SCIENCE AND INFORMATION TECHNOLOGY

B.TECH – AIDS 22PBAD51– JAVA AND WEB PROGRAMMING LABORATORY

ACADEMIC YEAR 2026-2027

NAME OF THE STUDENT :

REGISTER NUMBER :

COURSE :

YEAR :

SEMESTER :

VELS

INSTITUTE OF SCIENCE, TECHNOLOGY & ADVANCED STUDIES (VISTAS)
(Deemed to be University Estd. u/s 3 of the UGC Act, 1956)
PALLAVARAM, THALAMBUR, PERIYAPALAYAM, THIRUVANMIYUR - CHENNAI

BONAFIDE CERTIFICATE

Reg. No. | | | | | | | | |

This is to certify that the Bonafide Record of this Practical Work was completed by Mr/Ms…… of **B.TECH-ARTIFICIAL INTELLIGENCE AND DATA SCIENCE)** in the **22PBAD51 - Java and Web Programming Laboratory** during the academic year of **2026 -2027.**

HEAD OF THE DEPARTMENT **STAFF-IN-CHARGE**

Submitted for the Practical Examination held on ……..

INTERNAL EXAMINER **EXTERNAL EXAMINER**





INSTITUTE OF SCIENCE, TECHNOLOGY & ADVANCED STUDIES (VISTAS)
(Deemed to be University Estd. u/s 3 of the UGC Act, 1956)
PALLAVARAM, THALAMBUR, PERIYAPALAYAM, THIRUVANMIYUR - CHENNAI



INDEX

S. NO.	DATE	TITLE	PAGE NO.	MARKS	SIGNATURE
1		STATIC WEB PAGE WITH HTML ELEMENTS	4		
2		WEB PAGE VALIDATION WITH FORM CONTROLS USING DHTML	13		
3		WEB PAGE WITH ALL TYPES OF CASCADING STYLE SHEETS	17		
4		JAVA USING SERVLETS	21		
5		INVOKE SERVLETS FROM HTML	25		
6		THREE-TIER APPLICATION	29		
7		WEBSERVICES AND DATABASE FOR TRAVEL AGENT	37		
8		GET API DEVELOPMENT WITH SPRING BOOT	48		
9		POST API DEVELOPMENT	54		
10		UPDATING DATA IN DATABASE USING PUT API	61		
11		RESPONSE CODE 200, 400, 500 WITH SPRING BOOT	66		

EX. NO:1 DATE:	STATIC WEB PAGE WITH HTML ELEMENTS

AIM

To design the following static web pages required for online book store.

ALGORITHM

Step 1: Home page:- the static home page must contains three pages
Step 2: Top frame:- logo and college name and links to homepage, login page, registration page and catalogue page
Step 3: Left frame:- at least four links for navigation which will display the catalogue of Respective links
Step 4: Right frame:- the pages to links in the left frame must be loaded here initially it Contains the description of the website.

DESCRIPTION

In this program the entire web pages are created using basic HTML tags. Home page is divided into 3 frames by using <frameset> and <frame> tags. A frame is used to display a web page within a web page.

- HOME PAGE: The static home page must contain three frames.
- LOGIN PAGE
- CATOLOGUE PAGE: The catalogue page should contain the details of all the books available in the web site in a table.
- REGISTRATION PAGE

<frameset>

The <frameset> tag defines a frameset.
The <frameset> element holds one or more <frame> elements.
Each <frame> element can hold a separate document.
The <frameset> element specifies HOW MANY columns or rows there will be in the frameset, and HOW MUCH percentage/pixels of space will occupy each of them.

<frame>

The <frame> tag defines one particular window (frame) within a <frameset>.
Each <frame> in a <frameset> can have different attributes, such as border, scrolling, the ability to resize, etc

PROGRAM

home.html

```
<frameset rows="40%,*">
   <frame src="top.html" noresize scrolling="NO" name="topframe">
   <frameset cols="15%,*">
      <frame src="left.html" noresize scrolling="NO" name="leftframe">
      <frame src="right.html" noresize name="rightframe" scrolling="auto">
   </frameset>
</frameset>
top.html
<html>
 <head>
  <title>Top Frame</title>
 </head>
 <body bgcolor="YellowGreen ">
  <img src="images/logo1.png" width="125" height="115" align="left">
  <img src="images/cse.png" width="125" height="115" align="right">
  <center>
   <marquee bgcolor="yellow" width="650" behavior="alternate">
    <font face="Brush Script MT" size="8" color="green"><b><i>Online Book Store</i></b>
     </font>
   </marquee> <br>
   <font face="Brush Script" size="6" color="white"><b>Created & Maintained By
MRCET</b></font>
  </center>
  <br>
<table width="100%" height="50%" cellspacing=10>
<tr align="center">
      <td>  <a href="Home.html" target="_parent"><font face="Brush Script" size="6"
color="navy">HOME </a> </td>
      <td> <a href="login.html" target="rightframe"><font face="Brush Script" size="6"
color="navy">LOGIN</a> </td>
      <td> <a href="registration.html" target="rightframe"> <font face="Brush Script" size="6"
color="navy">REGISTER </a> </td>
      <td> <a href="catalogue.html" target="rightframe"> <font face="Brush Script" size="6"
color="navy">CATALOGUE</a> </td>
  </tr>
  </table>
 </body>
</html>
left.html
<html>
  <body align="center" bgcolor="bisque"> <br>
   <a href="cse.html" target="rightframe"><font size="6">CSE</font></a><br><br>
   <a href="ece.html" target="rightframe"><font size="6">ECE</font></a><br><br>
   <a href="eee.html" target="rightframe"><font size="6">EEE</font></a><br><br>
   <a href="mech.html" target="rightframe"><font size="6">MECH</font></a><br>
 </body>
```

```
</html>
right.html
<html>
  <body bgcolor="orange">
   <center>
    <img src="images/Books.jpg" height="170"><br>
    <font face="Brush Script MT" size="5" color="blue">
     <h1><b>Welcome to the Online Book Store!!!</b></h1><br />
    <font face="Brush Script MT" size="5" color="red">
     <h2><b> "A Huge Collection Of Engineering E-Books"</b></h2></font>
  </center>
 </body>
</html>
cse.html
<html>
   <head><title>CSE</title></head>
 <body bgcolor="cyan">
 <center><font color="blue"><h1>Computer Science and Engineering</h1></font></center>
 <br>
   <table align="center">
    <tr>
        <td>Text Books</td>
        <td>
        <select >
           <option value="select the book" selected>Select the book
           <option value="C&Ds">C&Ds
           <option value="Ads">Ads
           <option value="Java">Java
           <option value="Oracle">Oracle
           <option value="Ms SQL Server">Ms SQL Server
           <option value="MySql">MySql
        </select>
          </td></tr>
    <tr>
        <td>Quantity</td>
        <td><input type="text" id="q"></td>
    </tr>
    <tr>
        <td></td>
        <td>
           <form method=post action="order.html">
              <input type="submit" value=ok />
           </form>
        </td>
    </tr>
  </table>
  <center>
    <pre> Cost of one book is"500" + shipping "100" </pre>
  </center>
```

```
  </body>
</html>
```

ece.html

```
<html>
   <body bgcolor="Plum">
      <h1><font color="blue">Electronics and Communication Engineering</font></h1>
      <h2>
      <ul>
   <li>Digital Circuits</li>  <li>Signals and Systems</li> <li>Digital Communication</li>
      </ul>
      </h2>
   </body>
</html>
```

eee.html

```
<html>
   <body bgcolor="Plum">
      <h1><font color="blue">Electrical and Electronics Engineering</font></h1>
      <h2>
         <ul type="square">
            <li>Concepts in Electric Circuits</li>
            <li>Introduction to Electronic Engineering</li>
            <li>Electrical Power</li>
         </ul>
      </h2>
   </body>
</html>
```

mech.html

```
<html>
   <body bgcolor="Plum">
      <h1><font color="blue">Electronics and Communication Engineering</font></h1>
      <h2>
         <ol type="I">
                  <li>Theory of Machines</li>
                  <li>Automation and Robotics</li>
                  <li>Engineering Fluid Mechanics</li>
            </ol>
      </h2>
   </body>
</html>
```

catalogue.html

```
<html>
  <head>
      <title> Catalogue </title>
   </head>
<body bgcolor="pink">
  <form action="order.html">
   <table border="1" width="100%">
     <tr>
       <td>
          <img src="images/wt.jpg" width=100 height=100/>
       </td>
       <td> Book: Web Technologies <br> Author: Uttam K. Roy <br> Publication:Oxford
            University Press</td> <td>531    </td>
       <td> <input type="submit" value="Add to cart"/></td> </tr>
     <tr>
       <td> <img src="images/php.jpg" width=100 height=100/></td>
       <td> Book: PHP & MySQL Web Development <br> Author:Luke Welling & Laura
          Thompson <br> Publication:PEARSON</td> <td> 898     </td>
       <td> <input type="submit" value="Add to cart"/></td> </tr>
</table> </form>
</body> </html>
```

Login.html

```
<html>
  <body bg color="pink">
   <basefont face="Cambria" size="4"> <br>
   <center>
    <img src="images/login.jpg" width="385" height="135" /><br />
    <font face="Brush Script MT" size="7" color="purple">
      <b>Enter Login Details:</b>
    </font>
   </center>
  <form name="f1" method="post" action="right.html">
  <table align="center" width="100" height="150" cellspacing="15">
    <tr><td><b>Login ID:</b></td>
       <td><input type="text" name="t1"></td>
    </tr>
    <tr>
       <td><b>Password:</b></td>
       <td><input type="password" name="t2"></td>
    </tr>
    <tr align="center">
       <td><input type="submit" name="b1" value="Submit"></td>
       <td><input type="reset" name="b2" value="Reset"></td>
    </tr>
  </table>    </form>    </basefont> </body>      </html>
```

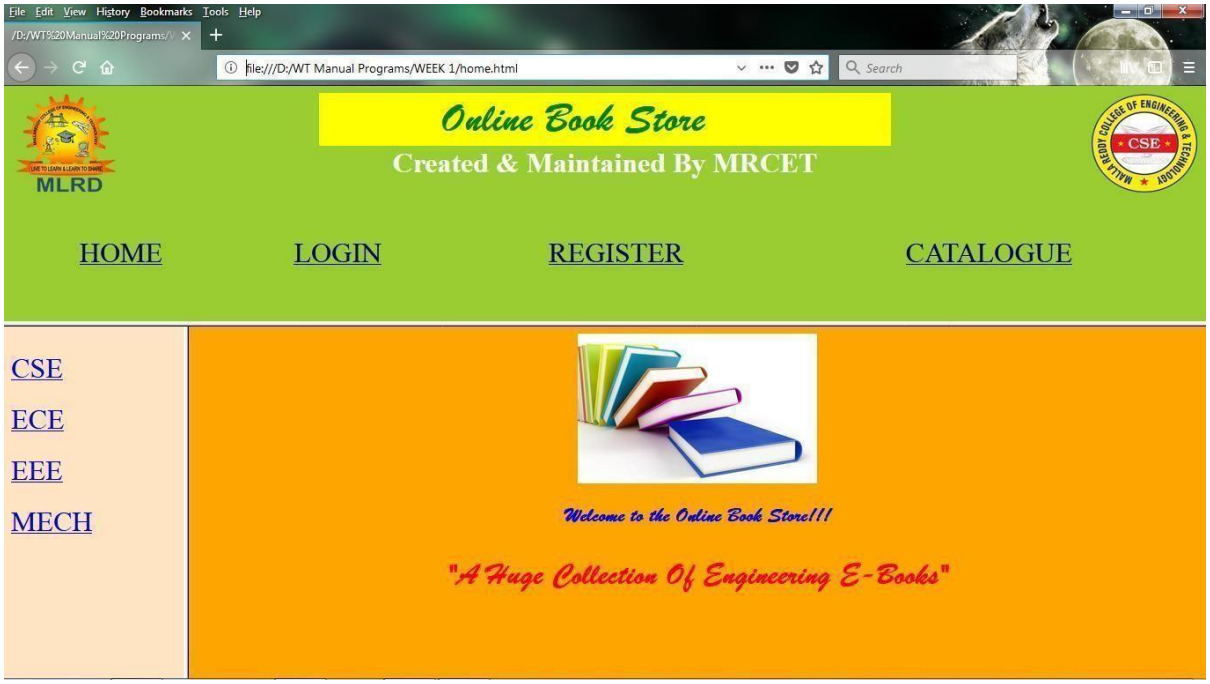
registration.html

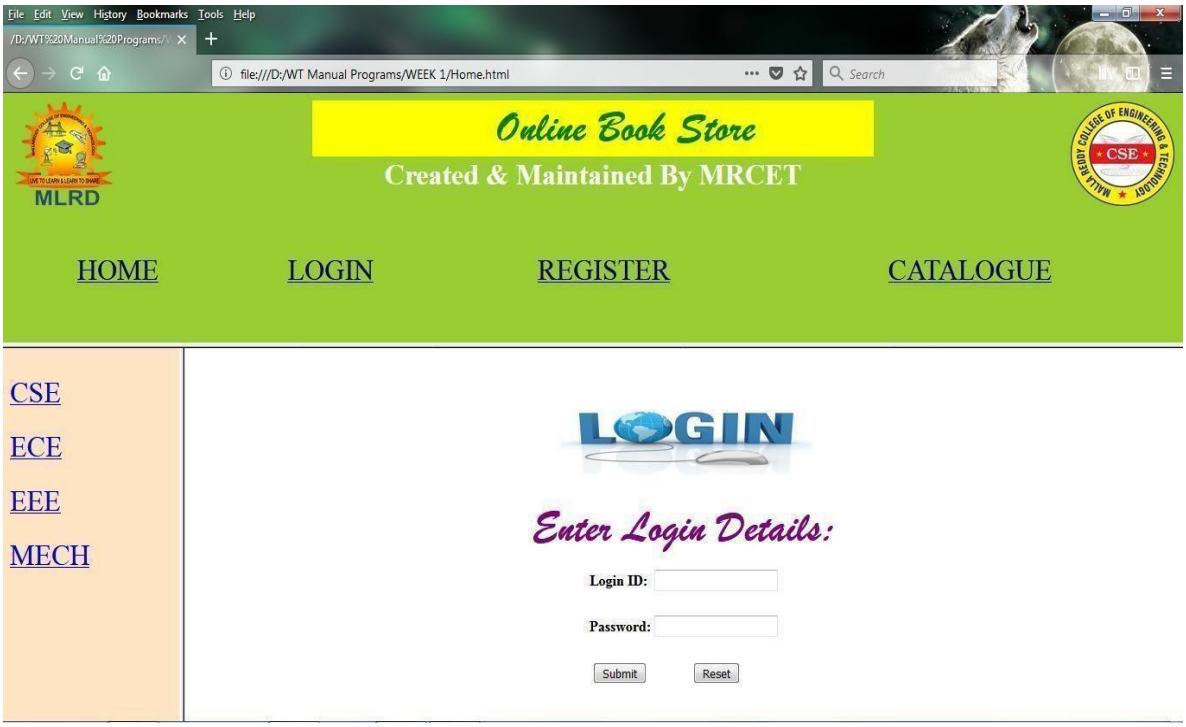
```
<html>
<head><title>Registration Form</title></head>
<body bgcolor="#E4F0F8">
<center><font color="blue" size="6" face="arial">Registration Form</font></center><br />
<form action="right.html">
   First Name(Minimum 6 characters)<font color="red">* </font>
     <input type='text' id='firstname' /><br /><br />
   Last Name<font color="red"><font color="red">* </font> </font>    
     <input type='text' id='lastname' /><br /><br />
   EmailAddress<font color="red">* </font>    
     <input type='text' id='email' /><br />
 <font color="red">(one e-mail id only):</font>    
 <font color="redblue">e.g. smith@hotmail.com</font><br /><br/>
   Password(minimum 6 characters)<font color="red">* </font>    
     <input type='password' id='pass'><br /><br/>
   Address<font color="red">* </font>    
     <textarea rows="2" cols="20" id='addr' /></textarea> <br /> <br/>
   Mobile No<font color="red">* </font>    
     <input type='text' id='mobileno' /><br />
   Gender: <input type='radio' name='gender'>male
           <input type='radio' name='gender'>female<br/><br />
     <input type='Submit' value='submit' />
     <input type='Reset' value='reset' />
</form>
</body>
 </html>
```

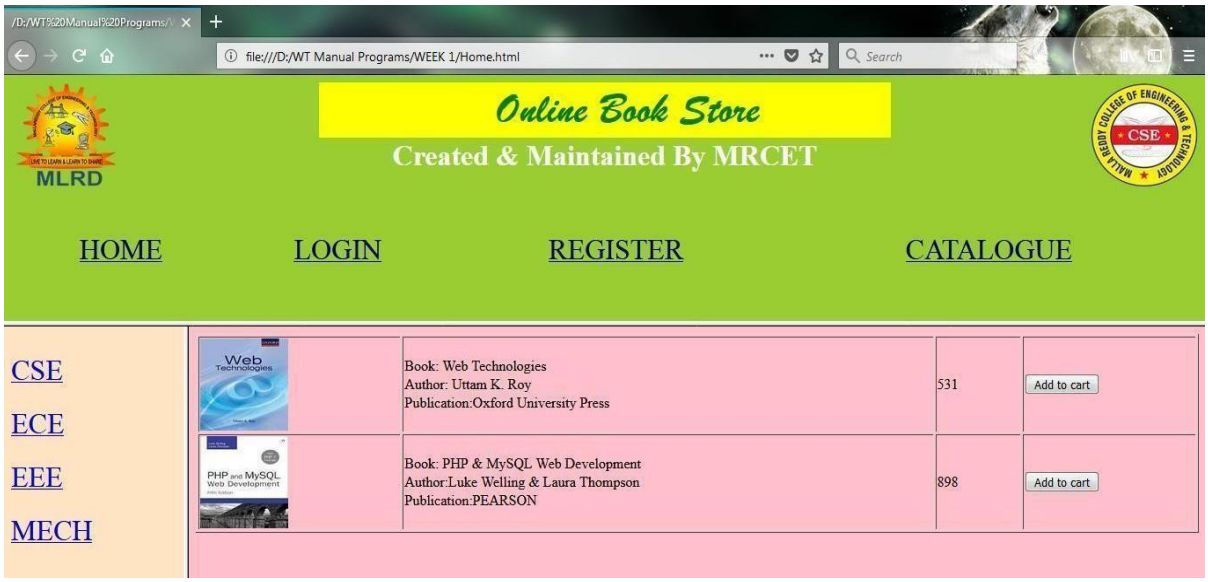
order.html

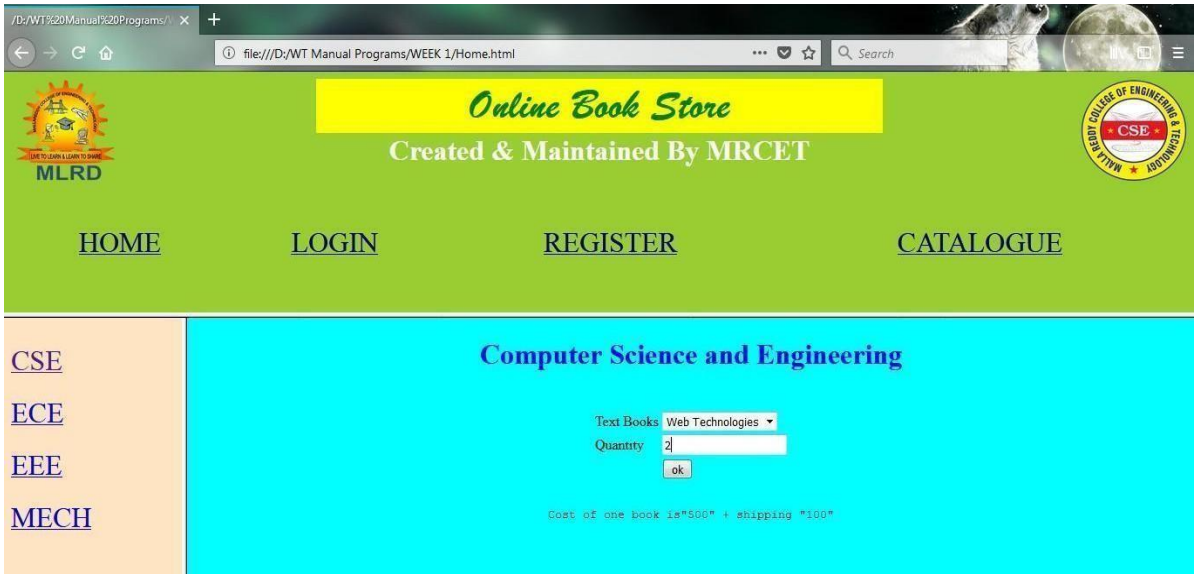
```
<html>
  <head><title>order conformation</title></head>
    <body bgcolor="cyan">
      <center>
        <pre><strong>
        <b>Your order Is Conformed
        </strong></pre>
        <h2><b>THANK YOU...Visit Again</h2>
    </center>
  </body>
</html>
```

OUTPUT

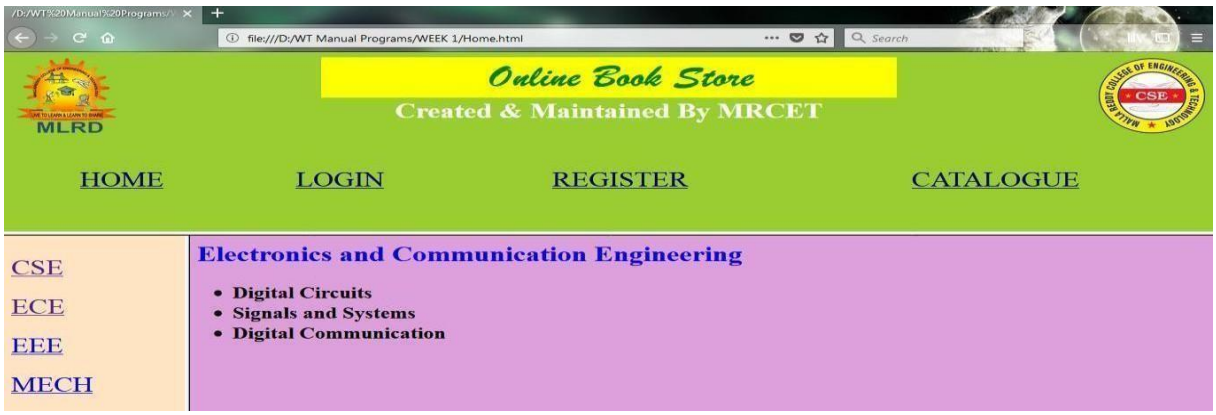

Online Book Store
Created & Maintained By MRCET
MLRD
HOME
LOGIN
REGISTER
CATALOGUE
CSE
ECE
EEE
MECH
Welcome to the Online Book Store!!!
"A Huge Collection Of Engineering E-Books"

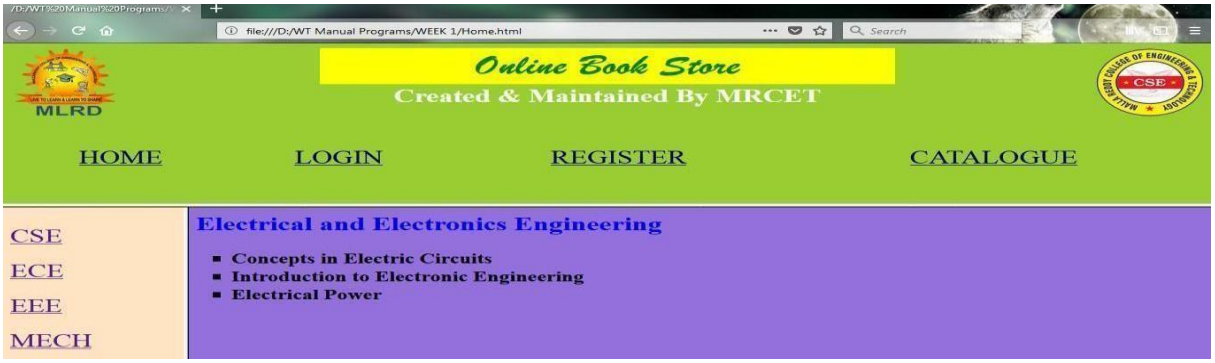

Online Book Store
Created & Maintained By MRCET
MLRD
HOME
LOGIN
REGISTER
CATALOGUE
CSE
ECE
EEE
MECH
LOGIN
Enter Login Details:
Login ID:
Password:
Submit
Reset

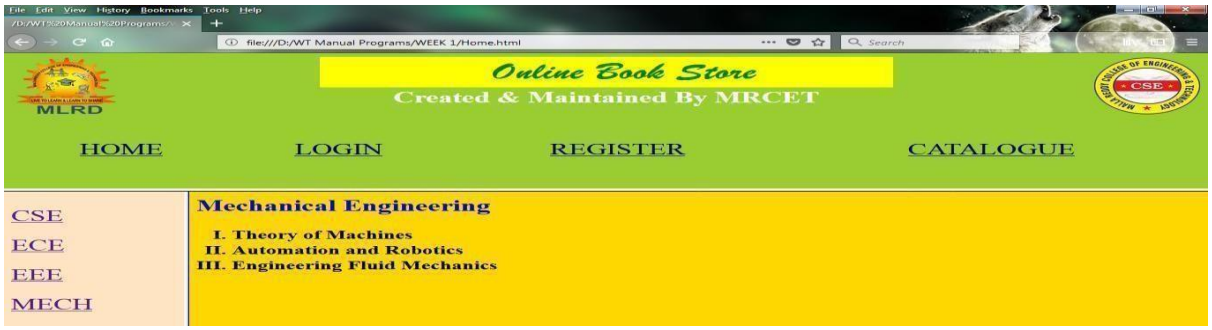

file:///D:/WT Manual Programs/WEEK 1/Home.html
Search
MLRD
Online Book Store
Created & Maintained By MRCET
CSE
HOME
LOGIN
REGISTER
CATALOGUE
CSE
ECE
EEE
MECH
Book: Web Technologies
Author: Uttam K. Roy
Publication:Oxford University Press
531
Add to cart
Book: PHP & MySQL Web Development
Author:Luke Welling & Laura Thompson
Publication:PEARSON
898
Add to cart


file:///D:/WT Manual Programs/WEEK 1/Home.html
Search
MLRD
Online Book Store
Created & Maintained By MRCET
CSE
HOME
LOGIN
REGISTER
CATALOGUE
CSE
ECE
EEE
MECH
Computer Science and Engineering
Text Books
Web Technologies
Quantity
2
ok
Cost of one book is"500" + shipping "100"


file:///D:/WT Manual Programs/WEEK 1/Home.html
Search
MLRD
Online Book Store
Created & Maintained By MRCET
CSE
HOME
LOGIN
REGISTER
CATALOGUE
CSE
ECE
EEE
MECH
Your order Is Conformed
THANK YOU...Visit Again


file:///D:/WT Manual Programs/WEEK 1/Home.html
Online Book Store
Created & Maintained By MRCET
MLRD
HOME
LOGIN
REGISTER
CATALOGUE
CSE
ECE
EEE
MECH
Electronics and Communication Engineering
Digital Circuits
Signals and Systems
Digital Communication


file:///D:/WT Manual Programs/WEEK 1/Home.html
Online Book Store
Created & Maintained By MRCET
MLRD
HOME
LOGIN
REGISTER
CATALOGUE
CSE
ECE
EEE
MECH
Electrical and Electronics Engineering
Concepts in Electric Circuits
Introduction to Electronic Engineering
Electrical Power


File Edit View History Bookmarks Tools Help
file:///D:/WT Manual Programs/WEEK 1/Home.html
Online Book Store
Created & Maintained By MRCET
MLRD
HOME
LOGIN
REGISTER
CATALOGUE
CSE
ECE
EEE
MECH
Mechanical Engineering
I. Theory of Machines
II. Automation and Robotics
III. Engineering Fluid Mechanics

RESULT

EX. NO: 2 DATE:	WEB PAGE VALIDATION WITH FORM CONTROLS USING DHTML

AIM

To Create a web page validation with Form Controls using DHTML

ALGORITHM

DHTML combines HTML, CSS, JavaScript, and the Document Object Model (DOM) to create dynamic content.

Step 1: Create html file (login form) using basic elements and form tag.

Step 2: Add Cascading style sheet to control the layout of web page.

Step 3: Validate the html form elements like username and password using java script.

- If true display welcome webpage.
- Else return to main login form webpage.

Step 4: Close </body> and </html> tag

PROGRAM

LOGIN FORM USING DHTML (myfile.html)

```
<html>
<head>
   <style>
     form{
       position: fixed;
       top: 50%;
       left: 50%;
       transform: translate(-50%, -50%);
       background-color: rgba(0, 0, 0, 0.5); /* semi-transparent background */
       padding: 20px;
       color: white;
       text-align: center;
     }

     input[type="text"],
     input[type="password"] {
       margin: 10px 0;
       padding: 5px;
     }

   </style>
 </head>
<body>

<form onsubmit="return formvalidate()" action="message.html">
   <input type="text" id="name" placeholder="Username">
   <input type="password" id="pwd" placeholder="Password"> <br>
   <button type="submit"> submit </button>
</form>

<script>

function formvalidate()
{
var nam=document.getElementById("name");
if(nam.value==""|| pwd.value=="")
{
alert("Name can't be blank");
return false;
}
else
{
return true;
}
}
```

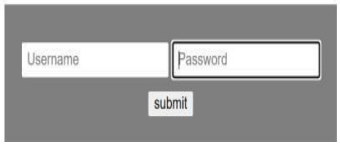
```
</script>
</body>
</html>
```

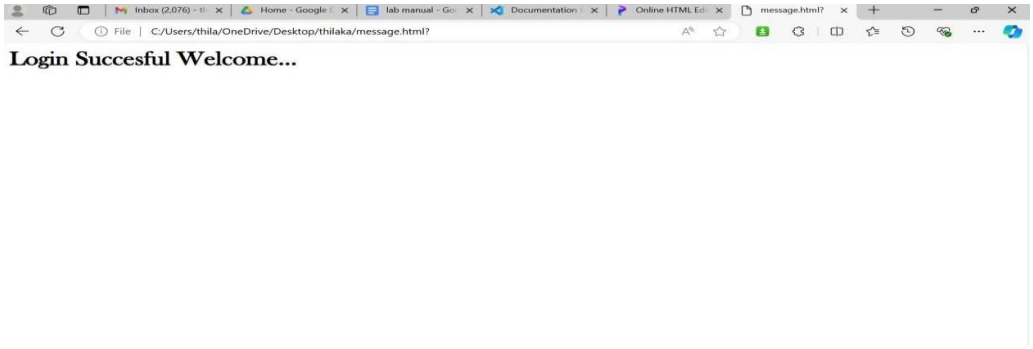
message.html

```
<html>
<head>
   <h1>Login Succesfull Welcome</h1>
</head>
</html>
```

OUTPUT







RESULT

EX. NO: 3 DATE:	WEB PAGE WITH ALL TYPES OF CASCADING STYLE SHEETS

AIM

To create a web page with all types of Cascading style sheets.

DESCRIPTION

CSS stands for Cascading Style Sheets.
CSS is used to format the layout of a webpage.
CSS CSS can be added to HTML documents in 3 ways:

- Inline - by using the style attribute inside HTML elements
- Internal - by using a <style> element in the <head> section
- External - by using a <link> element to link to an external CSS file

ALGORITHM

Step 1: Go to start- >all programs-> accessories->notepad
Step 2: Type the html code
Step 3: Use<style> tag for internal css to apply styles for body, h3 tags and close using</style>
Step 4: Use <style> tag for inline css in single element h2, p tags.

External CSS Steps:

- First, create the HTML file
- Create a CSS file and save it using the .css extension (This file only will only contain the styling code.)
- Link the CSS file in your HTML file using tag in header section of HTML document.

PROGRAM

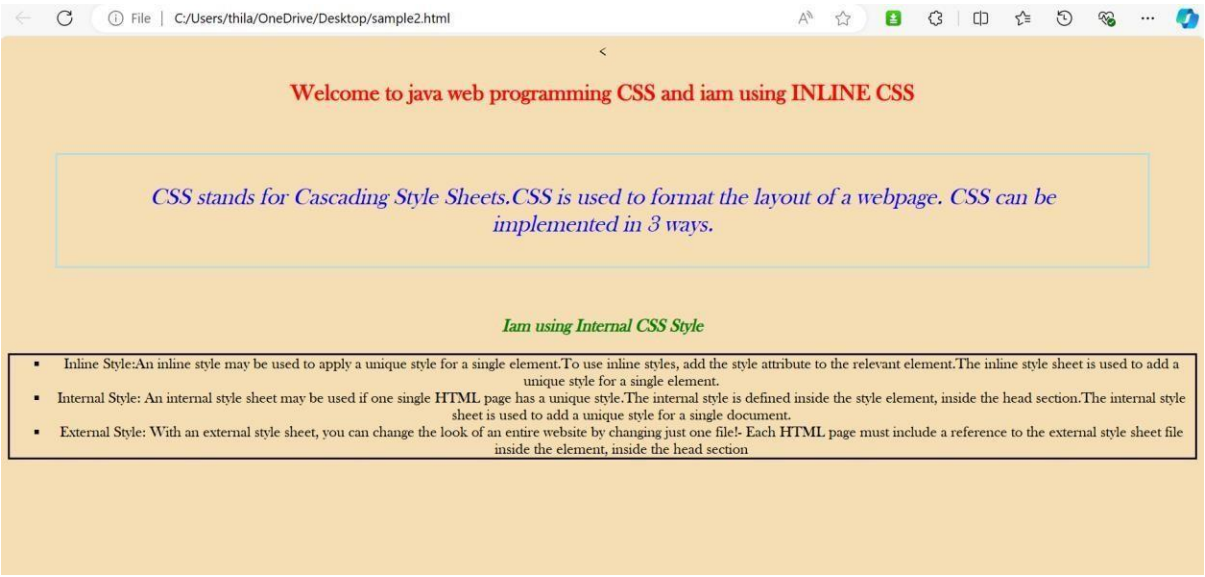
Sample.html

```
<html>
<head>
   <title>Types of CSS</title>
   <link rel="stylesheet" href="mystyle.css">
<style>
body
{
background-color:wheat;
}
h3
{
font-style: italic;
color: green;
}
</style>
</head>
<body style="text-align: center;">
   <h2 style="color: red;">Welcome to java web programming CSS and iam using INLINE CSS </h2>
   <p style="color: blue; font-size: 25px; font-style: italic;">CSS stands for Cascading Style
   Sheets.CSS is used to format the layout of a webpage.
   CSS can be implemented in 3 ways.</p>
<h3>Iam using Internal CSS Style</h3>
<ul>
  <li>Inline Style:An inline style may be used to apply a unique style for a single element.To
  use inline styles,
      add the style attribute to the relevant element.The inline style sheet is used to add a
  unique style for a single element.</li>
  <li>Internal Style: An internal style sheet may be used if one single HTML page has a
  unique style.The internal style is defined
      inside the style element, inside the head section.The internal style sheet is used to add a
  unique style for a single document.</li>
  <li>External Style: With an external style sheet, you can change the look of an entire
  website by changing just one file!-
      Each HTML page must include a reference to the external style sheet file inside the
  <link> element, inside the head section</li>
</ul>

</body>
</html>
mystyle.css
p{
border: 2px solid powderblue;
padding: 30px;
margin: 50px;
}
ul{
```

```
 border:2px solid black;
 align-items: left;
 list-style-type: square;
}
```

OUTPUT



RESULT

EX. NO: 4 DATE:	JAVA USING SERVLETS

AIM

To implement a web application using servlets.

ALGORITHM

Step 1: Create a dynamic project under Tomcat for your application.
Step 2: Open java class file and write the servlet source code save in the name of Myservlet.java.
Step 3: import the javax.servlet package and the javax.servlet.http package in your source file.
Step 4: Compile the source code by right clicking on the project-> run.
Step 5: Create a deployment descriptor web.xml with servlet class name and mapping information.
Step 6: Run Tomcat server.
Step 7: Call your servlet from a web browser (By default, Tomcat runs on port 8080).
URL: http://localhost:8080/program4/servlet1

Myservlet.java

```
package com.program4;

import javax.servlet.*;

import javax.servlet.http.*;

import java.io.*;

import java.io.IOException;

public class Myservlet extends HttpServlet

{

      public void doGet(HttpServletRequest req,HttpServletResponse res)throws
IOException,ServletException

            {

                  PrintWriter out=res.getWriter();

                  out.println("<html><head></head>");

                  out.println("<body bgcolor='lightgreen'>");

                  out.println("<h1>Hello iam tomcat server...</h1>");

                  out.println("<h1>Invoking Servlet...</h1>");

                  out.println("<h1>U too can See: type as below in ur browser.. make LAN
connect</h1>");

               out.println("<h1 style='color:red;'>192.168.175.254:8080/program4/servlet1</h1>");

                  out.println("</body></html>");

          }

}
```

web.xml

```
<?xml version="1.0" encoding="UTF-8"?>

<web-app xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns="http://xmlns.jcp.org/xml/ns/javaee"
xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee
http://xmlns.jcp.org/xml/ns/javaee/web-app_4_0.xsd" version="4.0">

  <display-name>program4</display-name>

  <description>
      Welcome to Tomcat
  </description>

  <servlet>

      <servlet-name>servlet1</servlet-name>

      <servlet-class>com.program4.Myservlet</servlet-class>

  </servlet>

  <servlet-mapping>

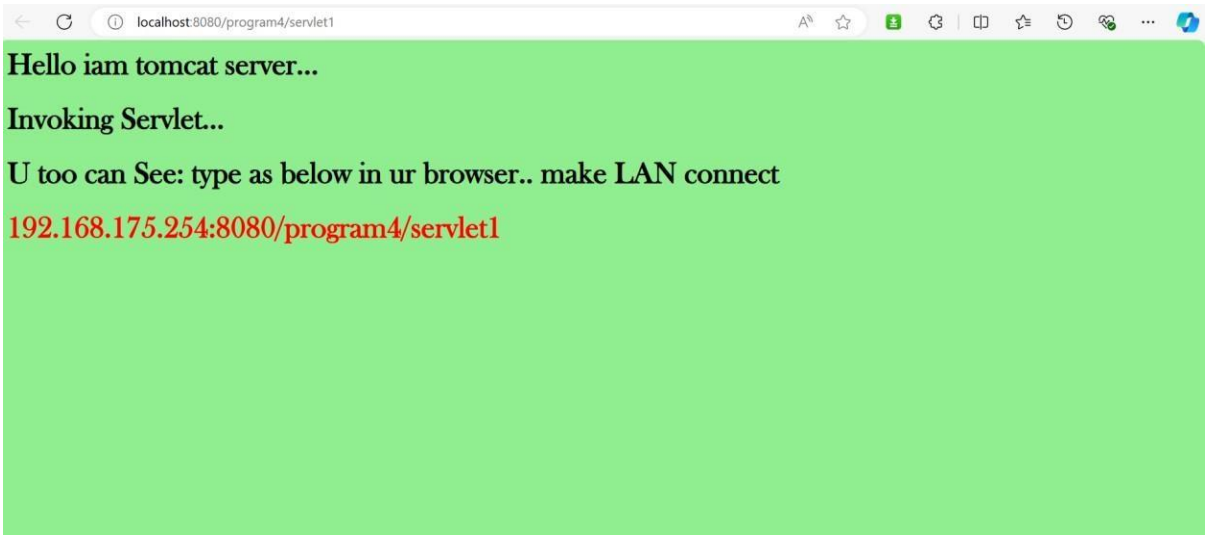
      <servlet-name>servlet1</servlet-name>

      <url-pattern>/servlet1</url-pattern>

  </servlet-mapping>

  </web-app>
```

OUTPUT



RESULT

EX. NO: 5 DATE:	INVOKE SERVLETS FROM HTML

AIM

To write a java program to invoke servlets from HTML form.

ALGORITHM

client.html:

Step 1: Create a web page using HTML form that contains the fields to collect two numbers and one submit button.
Step 2: Set the URL of the server as the value of the form's action attribute.
Step 3: Run the HTML program.
Step 4: Submit the form data to the server.

server.java:

Step 1: Define the class server that extends the property of the class HttpServlet
Step 2: Handle the request from the client by using the method service() of HttpServlet class.
Step 3: Get the parameter values from the HTML forms by using the method getParameter().
Step 4: Send the response to the client by using the method of PrintWriter class.

PROGRAM

index.html

```
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<h1 align="center">Addition Two Numbers using servlets</h1>
</head>
<body style="background-color:powderblue;">
<center>
<form action="add">
Enter first number:<input type="text" name="num1"><br>
Enter second number:<input type="text" name="num2"><br>
<input type="submit">
</form>
</center>
</body>
</html>
```

AddServlet.java

```
package com.myprog;

import java.io.IOException;

import java.io.PrintWriter;

import javax.servlet.http.HttpServlet;

import javax.servlet.http.HttpServletRequest;

import javax.servlet.http.HttpServletResponse;

public class AddServlet extends HttpServlet

	{

	public void service(HttpServletRequest req, HttpServletResponse res)throws IOException

	{

		int i=Integer.parseInt(req.getParameter("num1"));

		int j=Integer.parseInt(req.getParameter("num2"));

		int k=i+j;

		//to write in console
```

```
        System.out.println("result is "+k);

        PrintWriter out = res.getWriter();

        out.println("Result is "+k);

    }
    }
```

web.xml

```
<?xml version="1.0" encoding="UTF-8"?>

<web-app xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns="http://xmlns.jcp.org/xml/ns/javaee"
xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee
http://xmlns.jcp.org/xml/ns/javaee/web-app_4_0.xsd" version="4.0">

  <servlet>

      <servlet-name>abc</servlet-name>

      <servlet-class>com.myprog.AddServlet</servlet-class>

  </servlet>

  <servlet-mapping>

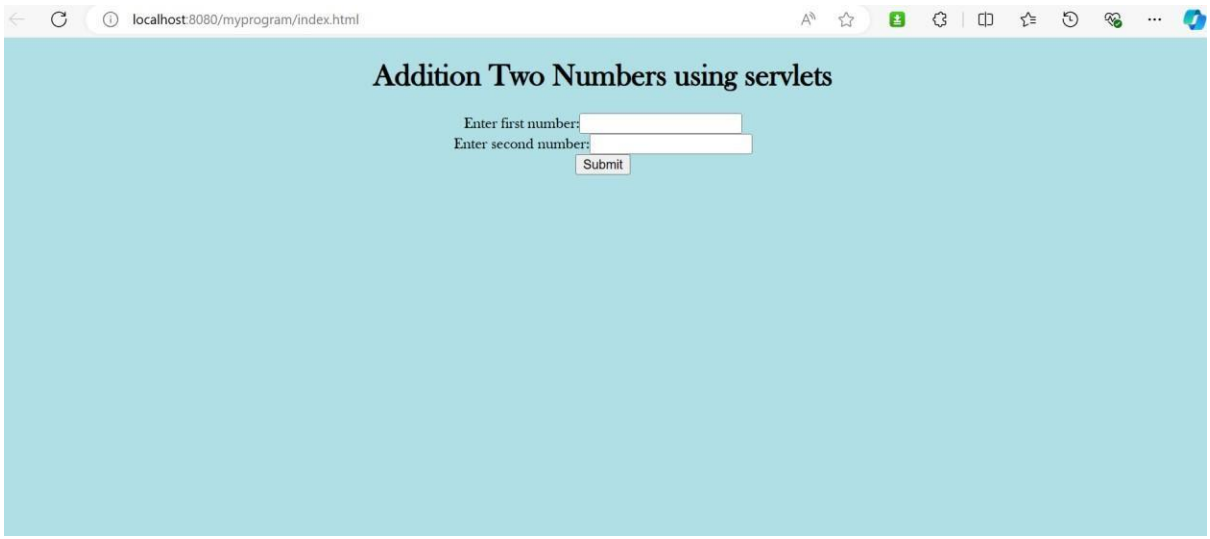
      <servlet-name>abc</servlet-name>

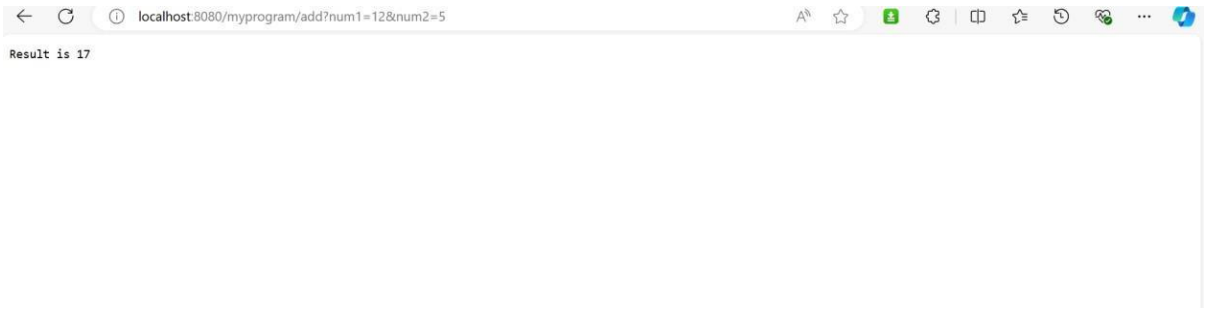
      <url-pattern>/add</url-pattern>

  </servlet-mapping>

</web-app>
```

OUTPUT





RESULT

EX. NO:6 DATE:	THREE-TIER APPLICATION

AIM

To develop three-tier applications using servlet, JSP and Databases.

TOOLS USED

- Eclipse
- MySql DB
- Apache Tomcat Server
- mysql connector jar file

PROCEDURE

MySQL

Create MySQL DB (database) and login table with fields <uname> <password>.

MySQL statements

```
create database mydb;  // to create database
use mydb; // DB in use
//table creation
create table login(uname varchar(20) primary key, password varchar(20));
insert into login values('Good','True'); // insert values in table
select * from login; // to display contents of the table
```

JSP

Create a Login form using a JSP page with a form tag and fields.
Text fields: <Username> <Password>
Buttons: <Login> <Reset>
On form submission call servlet class <form action=LoginServlet>

JAVA - SERVLET CLASS

Create a servlet class (LoginServlet.java) with the doPost method.
Include MySQL DB connection code in servlet class (default: MySQL port 3306).
Get HTML form parameters using getparameter method.
Prepare a query to check username and password with DB entries.
Execute query and get result set to proceed login success or fail.
Login Success-> if entered username and password exists in DB. Display Login successfull.
Login Failed->if entered username and password does not match with DB entries.

PROGRAM

Web.xml

```
<?xml version="1.0" encoding="UTF-8"?>

<web-app xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns="http://xmlns.jcp.org/xml/ns/javaee"
xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee
http://xmlns.jcp.org/xml/ns/javaee/web-app_4_0.xsd" id="WebApp_ID" version="4.0">

  <display-name>program6</display-name>

  <description>
      Welcome to Tomcat
  </description>

  <servlet>

      <servlet-name>loginservlet</servlet-name>

      <servlet-class>mypack.LoginServlet</servlet-class>

  </servlet>

  <servlet-mapping>

      <servlet-name>loginservlet</servlet-name>

      <url-pattern>/loginservlet</url-pattern>

  </servlet-mapping>

</web-app>
```

LoginServlet.java

```
package mypack;

import java.io.IOException;

import javax.servlet.RequestDispatcher;

import javax.servlet.ServletException;

import javax.servlet.annotation.WebServlet;

import javax.servlet.http.HttpServlet;
```

```
import javax.servlet.http.HttpServletRequest;

import javax.servlet.http.HttpServletResponse;

import java.sql.Connection;

import java.sql.DriverManager;

import java.sql.PreparedStatement;

import java.sql.ResultSet;

import java.io.PrintWriter;

import java.sql.SQLException;

/**
 * Servlet implementation class LoginServlet
 */

@WebServlet("/LoginServlet")
public class LoginServlet extends HttpServlet
{
	private static final long serialVersionUID = 1L;

	protected void doPost(HttpServletRequest request, HttpServletResponse response)
throws ServletException, IOException
	{
		try {

			PrintWriter out=response.getWriter();

			out.println("<h1>Trying to connect DB...</h1>");

			Class.forName("com.mysql.cj.jdbc.Driver");

Connection
con=DriverManager.getConnection("jdbc:mysql://localhost:3306/mydb","root","thilak");

			String n=request.getParameter("txtName");

			String p=request.getParameter("txtPwd");

			PreparedStatement ps=con.prepareStatement("select uname
from login where uname=? and password=?");
			ps.setString(1, n);

			ps.setString(2, p);
```

```
            ResultSet rs=ps.executeQuery();
            if(rs.next())
            {
                out.println("<h1>Connection to DB successful</h1>");
                out.println("<html><head></head>");
                out.println("<body bgcolor='lightgreen'>");
                out.println("<div align=center>");
                out.println("<h1>Login succes</h1>");
                out.println("</div>");
                out.println("</body></html>");
            }
            else
            {
                out.println("<html><head></head>");
                out.println("<body bgcolor='lightgreen'>");
                out.println("<div align=center>");
                out.println("<h1 style='color:red;'>Login failed</h1>");
                out.println("<h1 style='color:red;'>Try with Correct
Uname and Pwd</h1>");
                out.println("</div>");
                out.println("</body></html>");
            }
        }
        catch (ClassNotFoundException e)
        {
            e.printStackTrace();
        }
```

```
            catch (SQLException e)
            {
                  e.printStackTrace();
            }

      }
}
```

Login.jsp

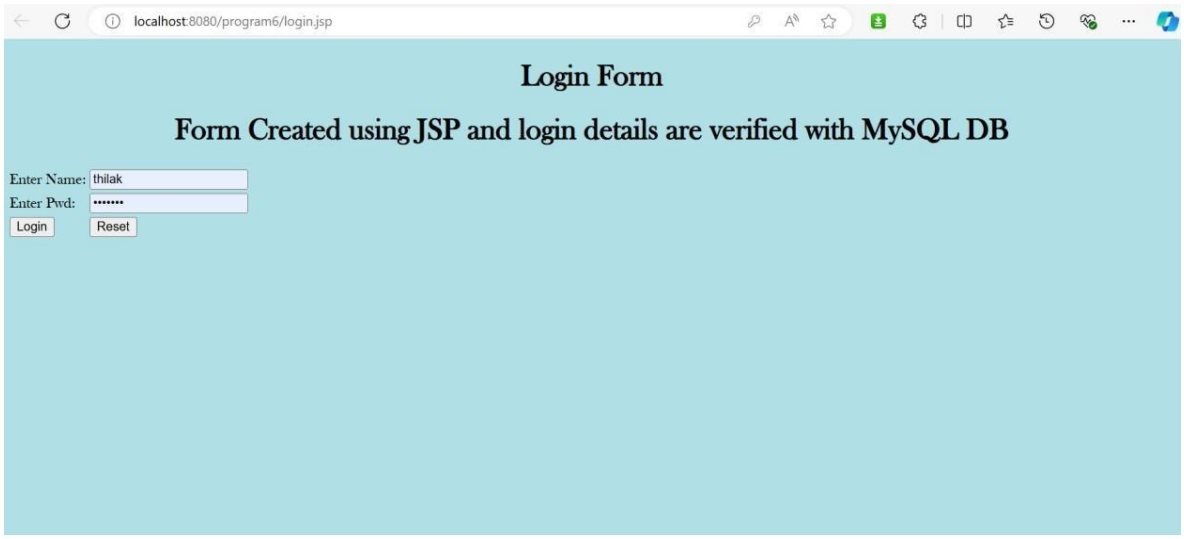
```
<html>
<head>
<meta charset="UTF-8">
<title>Login</title>
</head>
<body bgcolor=powderblue>
<div align=center>
<h1>Login Form</h1>
<h1>Form Created using JSP and login details are verified with MySQL DB</h1>
</div>
<form action=LoginServlet method=post>
<table>
<tr><td>Enter Name:</td><td><input type=text name=txtName></td></tr>
<tr><td>Enter Pwd:</td><td><input type=password name=txtPwd></td></tr>
<tr><td><input type=submit value=Login></td><td><input type=reset></td></tr>
</table>
</form>
</body>
</html>
```

PROGRAM RUN PROCEDURE:

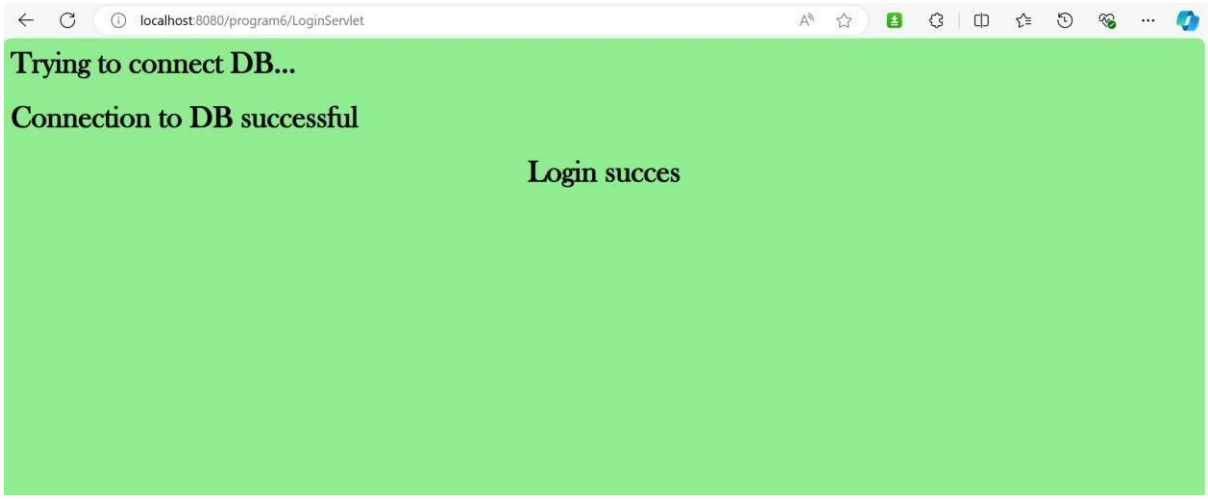
Download the recent version of mysql jar file from location: https://dev.mysql.com/downloads/connector/j/
Include mysql connector jar file in project->java->lib->runtime libraries.
Run JSP Page by right clicking and run on server (Tomcat server Started)
The Login form web page will be displayed on the browser.
Enter Username and password.
Click the Login button.
On successful login the page displays as login success else login fail is displayed.

OUTPUT

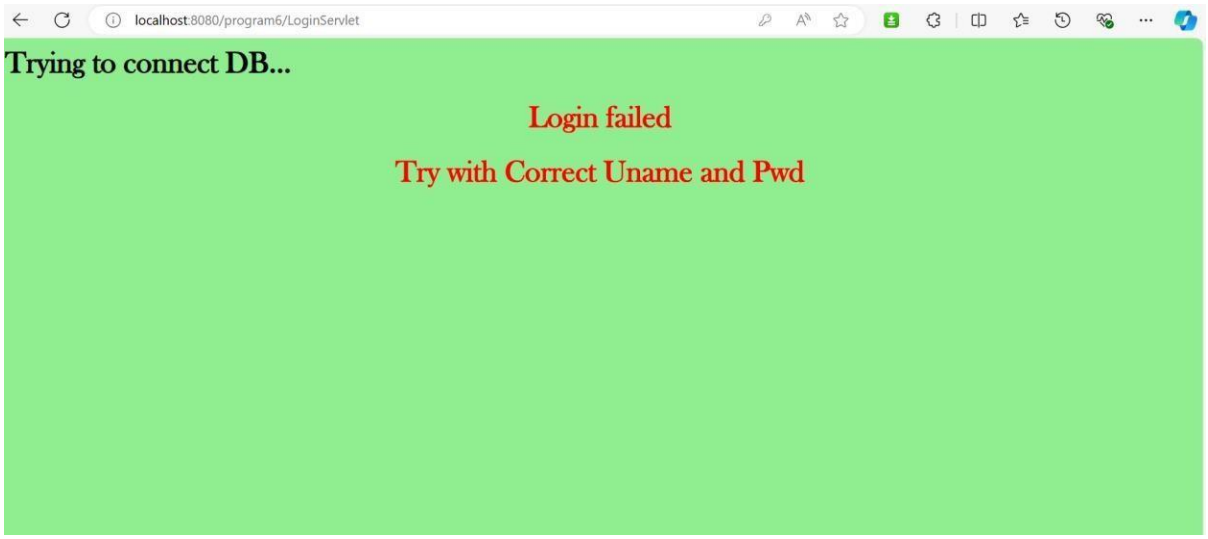
LOGINFORM WEBPAGE



LOGIN SUCCESS PAGE



LOGIN FAILED


localhost:8080/program6/LoginServlet
Trying to connect DB...
Login failed
Try with Correct Uname and Pwd

RESULT

EX. NO:7 DATE:	WEBSERVICES AND DATABASE FOR TRAVEL AGENT

AIM

To simulate the airline service and travel agent scenario using Web services and Database where the travel agent is searching for an airline.

ALGORITHM

Step 1: Open a new project in eclipse.
Step 2: Create a home page form using HTML tags in index.jsp with required fields.
Step 3: In project create all below jsp files by clicking New -> JSP
airlinelist.jsp file to collect all field details from the index home page.
Step 4: Establish JDBC connection to check flight availability on the mentioned date.
Step 5: Display airline details if flight available on scheduled date else display flights not available in searching date
Step 6: Collect passenger details to update the database.
Step 7: **Kingfisher.jsp:** Receive passenger details and update Database by reducing the passenger count
Step 8: **SpiceJet.jsp:** Receive passenger details and update Database by reducing the passenger count
Step 9: Run the project using Tomcat Server.

PROGRAM

index.jsp

```
<%@ page contentType="text/html" pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
 <head>
  <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
  <title>Travel Agency</title>
 </head>
 <body>
  <center><h1>Travel Easy</h1>
   <h3> - A Smarter way to travel</h3>
   <br><br>
   <b> Enter your Details</b>
   <form name="index" action="AirlineList.jsp" method="post">
    <br>
    <table cellpadding="6" cellspacing="6">
     <tr>
      <td><b>From</b></td>
      <td><input type="text" name="txt_from"></td>
     </tr>
     <tr>
      <td><b>To</b></td>
      <td><input type="text" name="txt_to"></td>
     </tr>
     <tr>
      <td><b>Date of Journey</b></td>
      <td><input type="text" name="txt_depart"></td>
     </tr>
     <tr>
      <td><b>Number of Passengers</b></td>
      <td><input type="text" name="txt_no"></td>
     </tr>
    <tr>
      <td><b>Type of Flight</b></td>
      <td><input type="Radio" name="group1" value="domestic">Domestic Flights</td>
    </tr>
    <tr>
      <td></td>
      <td><input type="Radio" name="group1" value="international">International Flights</td>
    </tr>
      <td><b>Select desired Airlines</b></td>
      <td>
       <select name="airline">
        <option>Kingfisher</option>
        <option>SpiceJet</option>
       </select>
      </td>
     </tr>
```

```
        </table>
        <br><br>
        <input type="Submit" name="Submit" value="Find Flights">
      </form>
    </center>
  </body>
</html>
```

airlinelist.jsp

```
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<!DOCTYPE html>

<html>
  <head>
    <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
    <title>Domestic Flights</title>
  </head>
  <body>
    <center><h1>Travel Easy</h1>
       <h3> - A Smarter way to travel</h3>
    </center>
    <br><br>
    <%@ page language="java" %>
    <%@ page import ="java.sql.*" %>
    <%@ page import =
"java.util.Date,java.text.SimpleDateFormat,java.text.ParseException"%>

    <%
    String type = request.getParameter("group1");
    String from= request.getParameter("txt_from");
    String to = request.getParameter("txt_to");
    String depart= request.getParameter("txt_depart");
    String ret = request.getParameter("txt_return");
    String no = request.getParameter("txt_no");
    String airline = request.getParameter("airline");
    Class.forName("com.mysql.cj.jdbc.Driver");
    String sSql = "SELECT * FROM stable WHERE Froms="'+from+"' and Tos="'+to+"'
and DepartDate="'+depart+"'";

    Connection cn = null;
    Statement st = null;
    ResultSet rs = null;
    try {
      cn = DriverManager.getConnection("jdbc:mysql://localhost:3306/airdb","root","thilak");
        st = cn.createStatement();
        rs = st.executeQuery(sSql);
        ResultSetMetaData rsmd = rs.getMetaData();
        String s1,s2,s3,s4,s5,s6,s7,s8;
```

```
        int i=1;
        String flight_no[] = new String[100];

        out.println("<h3><b>"+airline+" Flights</b></h3><br>");
        out.println("<table cellspacing=20 cellpaddinng=15>");
        out.println("<tr><th>Flight No</th><th>From</th><th>To</th><th>Departure
Date</th><th>Arrival Date</th><th>Start Time</th>");
        out.println("<th>Reach Time</th><th>Seats Remaining</th></tr>");
        while(rs.next())
        {
      s1 = rs.getString(1);
      flight_no[i] = s1;
      s2 = rs.getString(2);
      s3 = rs.getString(3);
      s4 = rs.getString(4);
      s5 = rs.getString(5);
      s6 = rs.getString(6);
      s7 = rs.getString(7);
      s8 = rs.getString(8);
      out.println("<tr><td>"+s1+"</td>");
      out.println("<td>"+s2+"</td>");
      out.println("<td>"+s3+"</td>");
      out.println("<td>"+s4+"</td>");
      out.println("<td>"+s5+"</td>");
      out.println("<td>"+s6+"</td>");
      out.println("<td>"+s7+"</td>");
      out.println("<td>"+s8+"</td></tr>");
      i++;
      }
      out.println("</table>");
      if(i==1)
      {
          out.println("<br><center><b>Sorry!! No flights scheudle
available</b></center><br>");
      }
      else
      {
        out.println("<form name='AirlineList' action='"+airline+".jsp' method='post'>");
        out.println("<br><table><tr><td>Select Flight</td><td>");
        out.println("<select name='flight_no'>");
        for(int temp=1;temp<i;temp++)
        {
          out.println("<option>"+flight_no[temp]+"</option>");
        }
        out.println("</select></td></tr><tr></tr>");
        out.println("<tr><td><input type='hidden' name='no' value='"+no+"'</td>");
        out.println("<td><input type='hidden' name='air_type'
value='"+type+"'</td></tr>");
        for(int j=1;j<=Integer.parseInt(no);j++)
```

```
        {
          out.println("<tr><th>Passenger "+j+ " details</th></tr>");
          out.println("<tr></tr>");
          out.println("<tr><td>Name</td><td><input type='text'
name='pas"+j+"_name'</td></tr>");
          out.println("<tr><td>Age</td><td><input type='text'
name='pas"+j+"_age'</td></tr>");
          out.println("<tr><td>Sex</td><td><input type='text'
name='pas"+j+"_sex'</td></tr>");
          out.println("<tr></tr>");
        }
       out.println("</table><br><br><input type='Submit' name='Submit' value='Book
Now'/></form>");
      }
      //out.println("<input name ='submit' value='Submit' type='submit'/>");
    }
    finally {
      try { if( null != rs ) rs.close(); } catch( Exception ex ) {}
      try { if( null != st ) st.close(); } catch( Exception ex ) {}
      try { if( null != cn ) cn.close(); } catch( Exception ex ) {}
    }
    %>
  </body>
</html>
```

Kingfisher.jsp

```
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
  <head>
    <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
    <title>Kingfisher Airlines</title>
  </head>
  <body>
    <%@ page language="java" %>
    <%@ page import ="java.sql.*" %>
    <%
    String s = request.getParameter("no");
    out.println("<h1><center>Kingfisher Airlines</h1><br><h3>- Have a nice
trip!!!</h3></center><br>");
    out.println("<br><br>Your Booking Details");
    out.println("<br><br><table cellspacing=10 cellpaddin=10>");
    Class.forName("com.mysql.cj.jdbc.Driver");
    String sTable = request.getParameter("air_type");
    String sSql = "SELECT * FROM stable WHERE
FlightNo='"+request.getParameter("flight_no")+"'";

    Connection cn = null;
```

```
    Statement st = null;
    ResultSet rs = null;
    String name,age,sex,flightno,from,to,departure,arrival,starttime,reachtime,seat;
    out.println("<tr><th>Name</th><th>Age</th><th>Sex</th><th>Flight
No</th><th>From</th><th>To</th><th>Departure Date</th><th>Arrival
Date</th><th>Start Time</th><th>Reach Time</th><th>Seat No</th></tr>");
    try
    {
    cn = DriverManager.getConnection("jdbc:mysql://localhost:3306/airdb","root","thilak");
      st = cn.createStatement();
      rs = st.executeQuery(sSql);
      ResultSetMetaData rsmd = rs.getMetaData();
      int seat1=1;
      while(rs.next())
      {
        flightno = rs.getString(1);
        from = rs.getString(2);
        to = rs.getString(3);
        departure = rs.getString(4);
        arrival = rs.getString(5);
        starttime = rs.getString(6);
        reachtime = rs.getString(7);
        seat = rs.getString(8);
        seat1 = Integer.parseInt(seat);
        for(int i=1;i<=Integer.parseInt(s);i++)
      {
        name= request.getParameter("pas"+i+"_name");
        age = request.getParameter("pas"+i+"_age");
        sex = request.getParameter("pas"+i+"_sex");
        flightno = request.getParameter("flight_no");

        out.println("<tr><td>"+name+"</td><td>"+age+"</td><td>"+sex+"</td>");
        out.println("<td>"+flightno+"</td><td>"+from+"</td><td>"+to+"</td><td>"+departure+"</
td>");
        out.println("<td>"+arrival+"</td><td>"+starttime+"</td><td>"+reachtime+"</td><td>"+seat
1+"</td>");
          seat1--;
        }
       }
       Connection cn1 = null;
       Statement st1 = null;
       ResultSet rs1 = null;
      try
      {
        cn1 = DriverManager.getConnection("jdbc:mysql://localhost:3306/airdb","root","thilak");
        String flightnum=request.getParameter("flight_no");
        PreparedStatement pst=cn1.prepareStatement("UPDATE stable SET
SeatCapacity=? WHERE FlightNo=?");
        pst.setInt(1,seat1);
```

```
        pst.setString(2,flightnum);
        pst.executeUpdate();
        out.println("Update success");
       }
       catch(Exception e)
       {
       }
    }
    finally {
     try { if( null != rs ) rs.close(); } catch( Exception ex ) {}
     try { if( null != st ) st.close(); } catch( Exception ex ) {}
     try { if( null != cn ) cn.close(); } catch( Exception ex ) {}
    }
    %>
  </body>
</html>
```

SpiceJet.jsp

```
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
  <head>
    <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
    <title>SpiceJet Airlines</title>
  </head>
  <body>
    <%@ page language="java" %>
    <%@ page import ="java.sql.*" %>
    <%
    String s = request.getParameter("no");
    out.println("<h1><center>SpiceJet Airlines</h1><br><h3>- Have a nice
trip!!!</h3></center><br>");
    out.println("<br><br>Your Booking Details");
    out.println("<br><br><table cellspacing=10 cellpaddin=10>");
    Class.forName("com.mysql.cj.jdbc.Driver");
    String sTable = request.getParameter("air_type");
    String sSql = "SELECT * FROM stable WHERE
FlightNo='"+request.getParameter("flight_no")+"'";

    Connection cn = null;
    Statement st = null;
    ResultSet rs = null;
    String name,age,sex,flightno,from,to,departure,arrival,starttime,reachtime,seat;
    out.println("<tr><th>Name</th><th>Age</th><th>Sex</th><th>Flight
No</th><th>From</th><th>To</th><th>Departure Date</th><th>Arrival
Date</th><th>Start Time</th><th>Reach Time</th><th>Seat No</th></tr>");
    try
    {
```

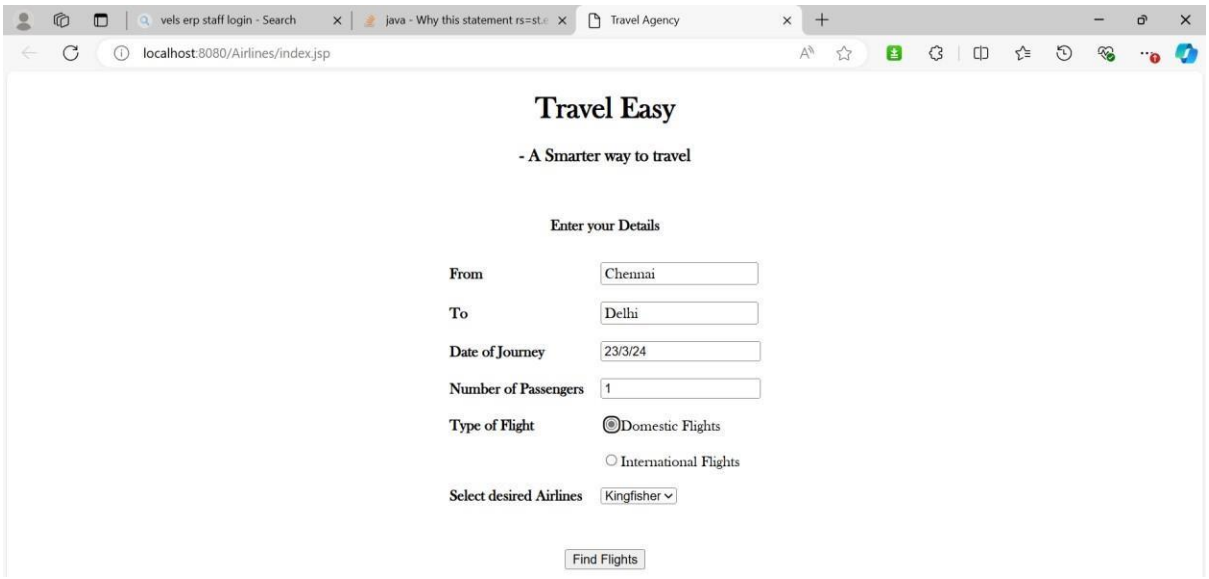
```
   cn = DriverManager.getConnection("jdbc:mysql://localhost:3306/airdb","root","thilak");
     st = cn.createStatement();
     rs = st.executeQuery(sSql);
     ResultSetMetaData rsmd = rs.getMetaData();
     int seat1=1;
     while(rs.next())
     {
       flightno = rs.getString(1);
       from = rs.getString(2);
       to = rs.getString(3);
       departure = rs.getString(4);
       arrival = rs.getString(5);
       starttime = rs.getString(6);
       reachtime = rs.getString(7);
       seat = rs.getString(8);
       seat1 = Integer.parseInt(seat);
       for(int i=1;i<=Integer.parseInt(s);i++)
     {
       name= request.getParameter("pas"+i+"_name");
       age = request.getParameter("pas"+i+"_age");
       sex = request.getParameter("pas"+i+"_sex");
       flightno = request.getParameter("flight_no");

       out.println("<tr><td>"+name+"</td><td>"+age+"</td><td>"+sex+"</td>");
       out.println("<td>"+flightno+"</td><td>"+from+"</td><td>"+to+"</td><td>"+departure+"</
td>");
       out.println("<td>"+arrival+"</td><td>"+starttime+"</td><td>"+reachtime+"</td><td>"+seat
1+"</td>");
         seat1--;
       }
      }
      Connection cn1 = null;
      Statement st1 = null;
      ResultSet rs1 = null;
     try
     {
       cn1 = DriverManager.getConnection("jdbc:mysql://localhost:3306/airdb","root","thilak");
       System.out.println("inside tryies loop"+seat1);
       String flightnum=request.getParameter("flight_no");
       PreparedStatement pst=cn1.prepareStatement("UPDATE stable SET SeatCapacity=?
WHERE FlightNo=?");
       pst.setInt(1,seat1);
       pst.setString(2,flightnum);
       pst.executeUpdate();
       out.println("Update success");
     }
     catch(Exception e)
     {
     }
```

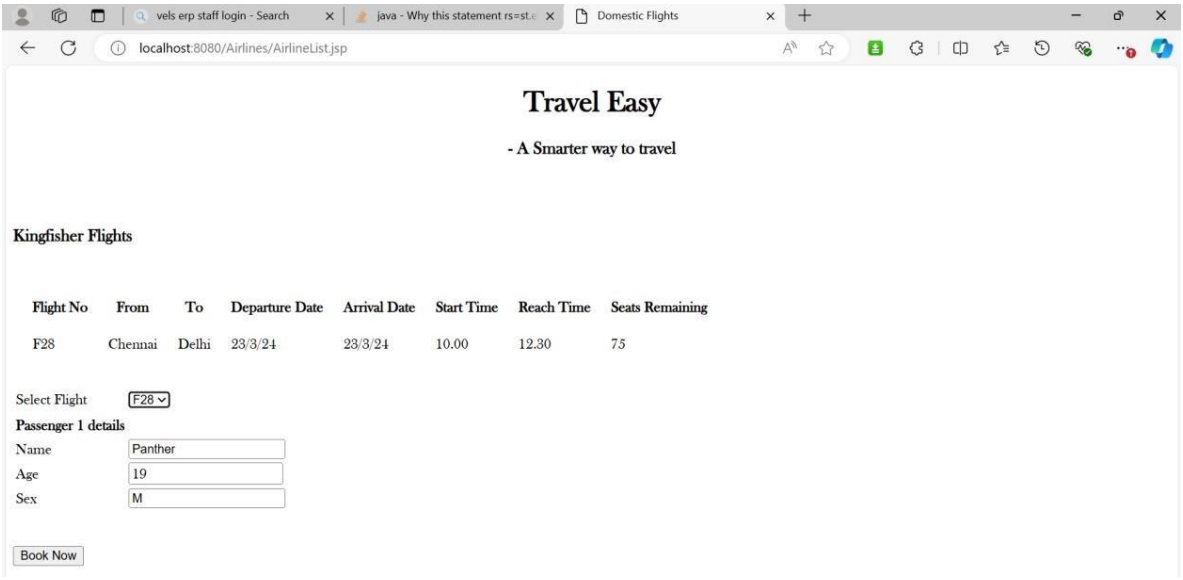
```
    }
    finally {
      try { if( null != rs ) rs.close(); } catch( Exception ex ) {}
      try { if( null != st ) st.close(); } catch( Exception ex ) {}
      try { if( null != cn ) cn.close(); } catch( Exception ex ) {}
    }
    %>
  </body>
</html>
```

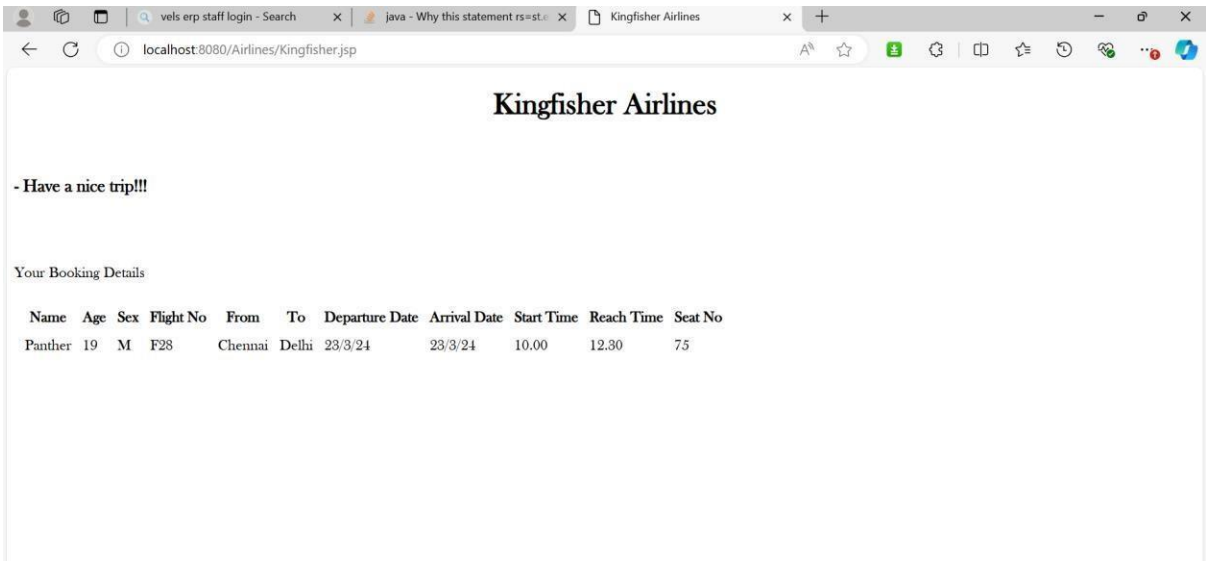
OUTPUT

HOME PAGE



KINGFISHER AIRLINES





AFTER BOOKING DB UPDATE (MYSQL CLI)

```
mysql> select * from stable;
+----------+---------+-------+------------+-------------+-----------+-----------+--------------+
| FlightNo | Froms   | Tos   | DepartDate | ArrivalDate | StartTime | ReachTime | SeatCapacity |
+----------+---------+-------+------------+-------------+-----------+-----------+--------------+
| F28      | Chennai | Delhi | 23/3/24    | 23/3/24     | 10.00     | 12.30     |           75 |
| F29      | Chennai | Delhi | 26/3/24    | 26/3/24     | 10.00     | 12.30     |           82 |
+----------+---------+-------+------------+-------------+-----------+-----------+--------------+
2 rows in set (0.00 sec)

mysql> select * from stable;
+----------+---------+-------+------------+-------------+-----------+-----------+--------------+
| FlightNo | Froms   | Tos   | DepartDate | ArrivalDate | StartTime | ReachTime | SeatCapacity |
+----------+---------+-------+------------+-------------+-----------+-----------+--------------+
| F28      | Chennai | Delhi | 23/3/24    | 23/3/24     | 10.00     | 12.30     |           74 |
| F29      | Chennai | Delhi | 26/3/24    | 26/3/24     | 10.00     | 12.30     |           82 |
+----------+---------+-------+------------+-------------+-----------+-----------+--------------+
2 rows in set (0.00 sec)

mysql>
```

RESULT

EX. NO:8 DATE:	GET API DEVELOPMENT WITH SPRING BOOT

AIM

To implement Get API development in spring boot installer.

PROCEDURE

Prerequisite

Install Java Development Kit (JDK) version 17 or newer.
To check, simply go to the terminal and enter the below command to check if it is present or not.
javac –version

ALGORITHM

Spring Boot Installation

Step 1: Install Spring boot from https://spring.io/tools in order to download. choose your corresponding file according to your OS.
Step 2: After clicking on the button a Jar file will be downloaded to your local system. Now create a folder and move this Jar file to that folder. And double-click on that Jar file.responding file according to your OS.
Step 3: After successfully Unpacking a new folder will be created as sts-4.12.0 RELEASE open the folder
Step 4: In this folder now click on the SpringToolSuite4 Application file.
Step 5: Now select your directory as workspace by clicking on the Browse button and then click on the Launch button.

To Create a Simple Spring Boot Project using STS.

Step 1: Create a new Spring project Go to the File > New > Maven Project.
Step 2: In the pop-up window fill the detail as per your project and further click on the Next button.
Step 3: Choose your required Spring Boot Version and select your dependencies as per your project requirement. And at last click on the Next button.
Step 4: Now click on the Finish button.

The CRUD operations refer to all major functions that are implemented in relational database applications.

Operation	SQL	HTTP verbs	RESTful Web Service
Create	INSERT	PUT/POST	POST
Read	SELECT	GET	GET
Update	UPDATE	PUT/POST/PATCH	PUT
Delete	DELETE	DELETE	DELETE

PROGRAM

Project hierarchy: simpleApp (boot) → src/main/java → com.simpleApp → HomeController.java, Product.java, ProductService.java, simpleApp.java; src/main/resources; pom.xml

simpleApp.java

```
package com.simpleApp;

import org.springframework.boot.SpringApplication;

import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class simpleApp
{
	public static void main(String[] args)
	{
		SpringApplication.run(simpleApp.class,args);
	}
}
```

Product.java

```
package com.simpleApp;
//Model Class
public class Product
{
	private int prodid;
	private String prodname;
	private int price;

	public Product()
	{
		//Default Constructor
	}
	public Product(int prodid, String prodname, int price) {
		super();
		this.prodid = prodid;
		this.prodname = prodname;
		this.price = price;
	}
	public int getProdid() {
		return prodid;
	}
	public void setProdid(int prodid) {
		this.prodid = prodid;
	}
	public String getProdname() {
		return prodname;
```

```
	}
	public void setProdname(String prodname) {
		this.prodname = prodname;
	}
	public int getPrice() {
		return price;
	}
	public void setPrice(int price) {
		this.price = price;
	}
}
```

HomeController.java

```
package com.simpleApp;

import java.util.List;

import org.springframework.beans.factory.annotation.Autowired;

import org.springframework.web.bind.annotation.RequestMapping;

import org.springframework.web.bind.annotation.RestController;

@RestController
public class HomeController
{
	@Autowired
	private ProductService service;

	@RequestMapping("/")
	public String greet()
	{
		return "welcome";
	}
	@RequestMapping("/about")
	public String aboutpage()
	{
		return "Iam about myself";
	}
	@RequestMapping("/products")
	public List<Product> getProducts()
	{
		return service.getProducts();
	}
}
```

ProductService.java

```
package com.simpleApp;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;
import org.springframework.stereotype.Service;
import com.simpleApp.Product;

@Service
public class ProductService
{
	private List<Product> products=new ArrayList<>(
			Arrays.asList(
					new Product(101,"Iphone",50000),
					new Product(102,"OnePlus",20000),
					new Product(103,"Redmi",30000)
			)
			);

	public List<Product> getProducts()
	{
		return products;

	}
}
```

pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE xml>
<project xmlns="https://maven.apache.org/POM/4.0.0"
xmlns:xsi="https://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="https://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>simpleApp</groupId>
  <artifactId>simpleApp</artifactId>
  <version>0.0.1-SNAPSHOT</version>
  <name>simpleApp</name>

<parent>
   <groupId>org.springframework.boot</groupId>
   <artifactId>spring-boot-starter-parent</artifactId>
   <version>2.2.6.RELEASE</version>
</parent>

<dependencies>
```

```
<dependency>
   <groupId>org.springframework.boot</groupId>
   <artifactId>spring-boot-starter-web</artifactId>
</dependency>
</dependencies>
<properties>
      <java-version>22.0.1</java-version>
</properties>
 </project>
```

OUTPUT

Run Project-> Type localhost:8080/



Run Project-> Type localhost:8080/about



Run Project-> Type localhost:8080/products



RESULT

EX. NO:9 DATE:	POST API DEVELOPMENT

AIM

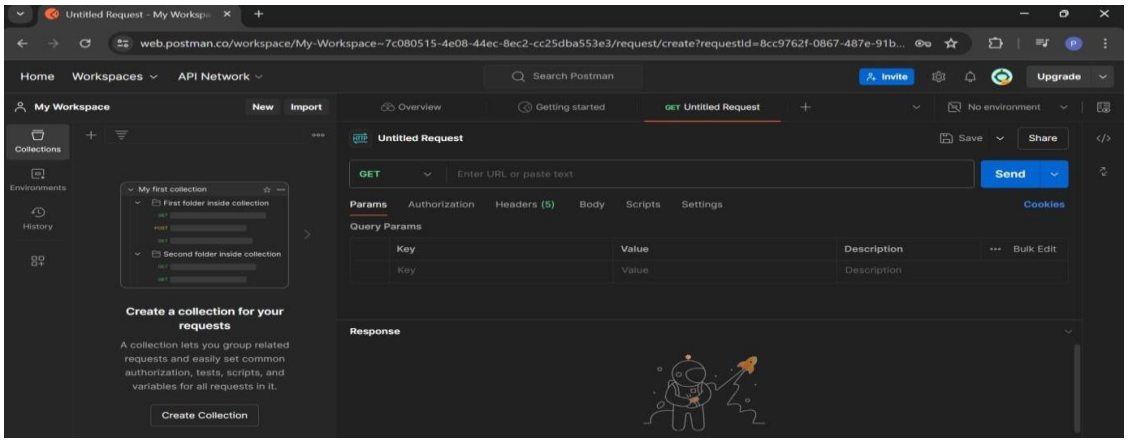
To implement POST API development in spring boot using Postman agent.

PROCEDURE

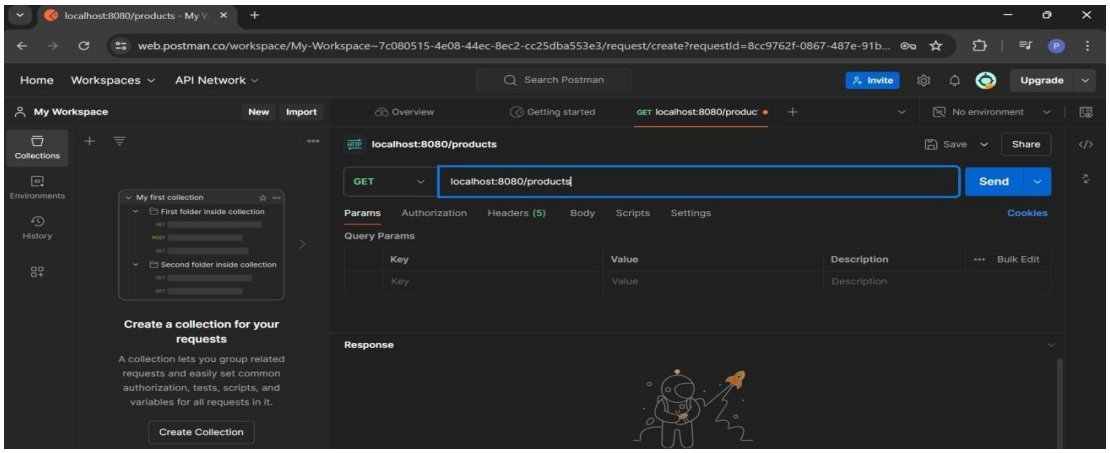
POST API - To store data on server POST API is used.

POSTMAN Installation:

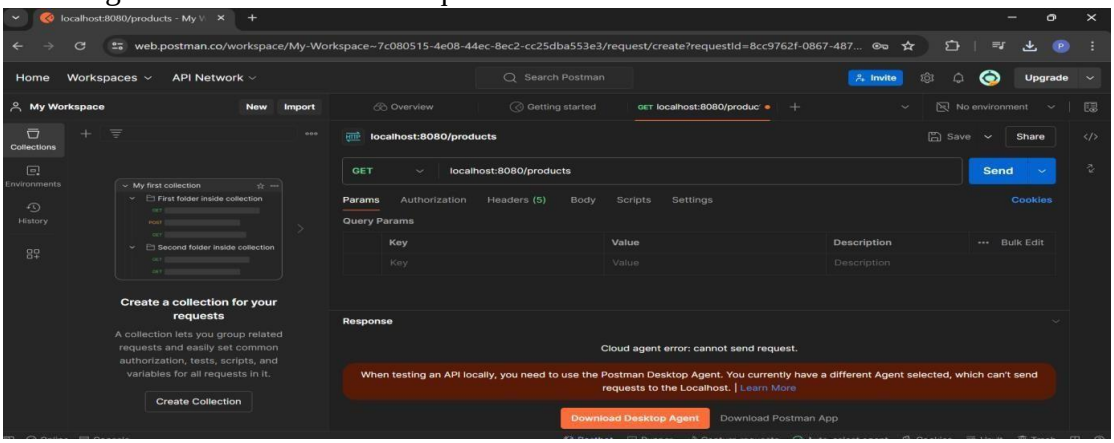
- Open browser and type postman.com/home — it is the official website of postman (Tool).
- Create a postman account.
- Restart the server in spring boot installer.



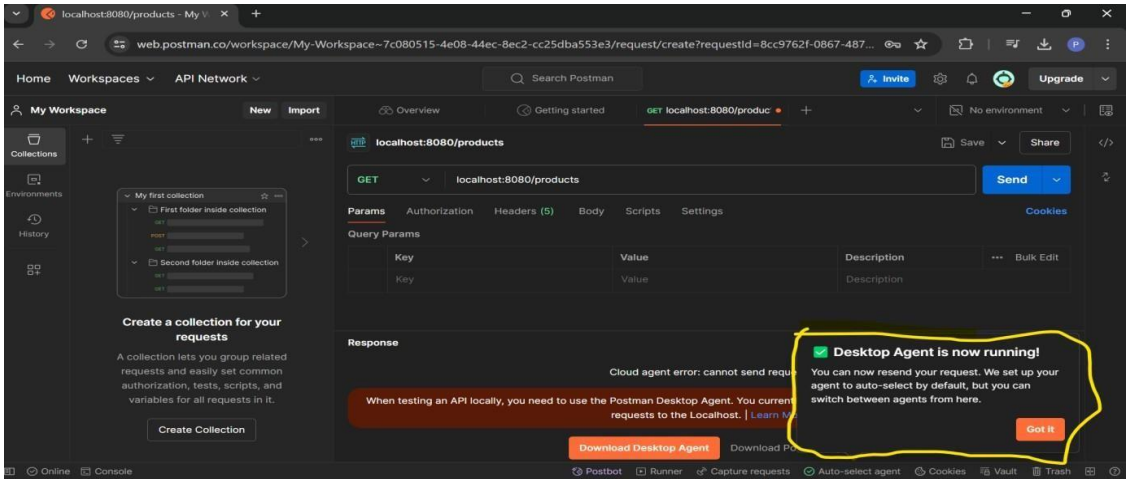
- In the official POSTMAN Site visit GET tab and type localhost:8080/products, click Send.



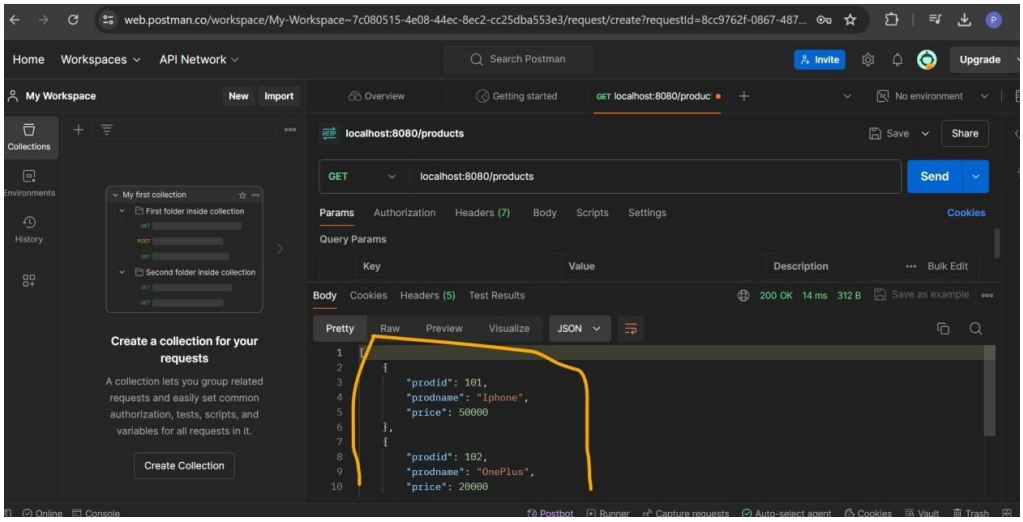
- If "Cloud agent error: cannot send request"



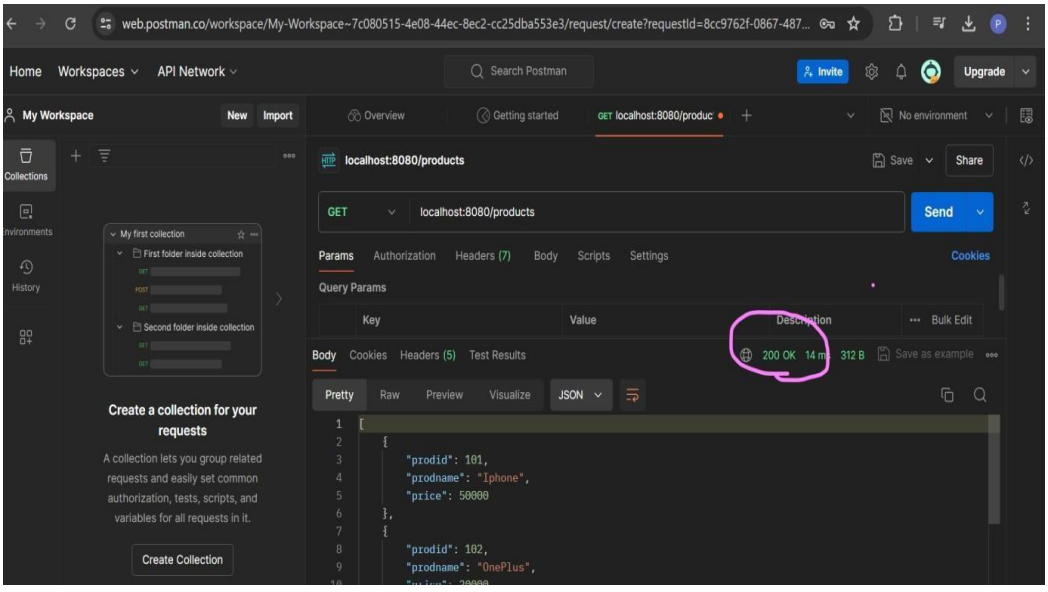
- You need to Download postman agent **setup exe and run (Make sure agent running message appears as shown below)**



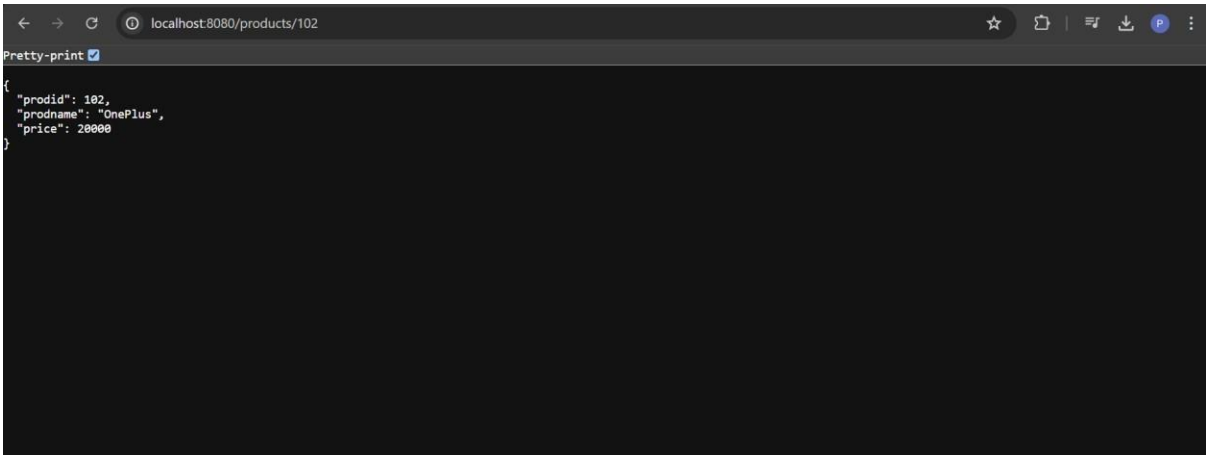
- Resend the request once "Desktop Agent is now running!" is confirmed



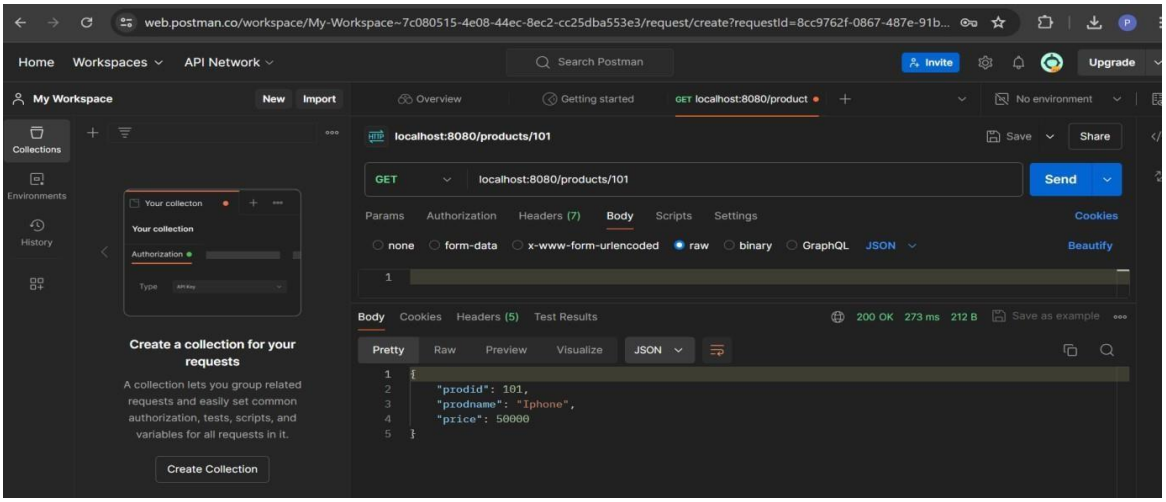
- The response now displays the product list with Status Code 200 OK.



- To display only one item (e.g. item 102), request localhost:8080/products/102 — in the browser or in Postman, both return {"prodid":102,"prodname":"OnePlus","price":20000}



Run in postman for individual element



PROGRAM

To Add Product in Item List: Write addProduct function in controller and service layer to update list with new item.

ProductService.java (updated)

```
package com.simpleApp;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;
import org.springframework.stereotype.Service;
import com.simpleApp.Product;

@Service
public class ProductService
{
	private List<Product> products=new ArrayList<>(Arrays.asList(
				new Product(101,"Iphone",50000),
				new Product(102,"OnePlus",20000),
				new Product(103,"Redmi",30000)
		)
		);

	public List<Product> getProducts()
	{
		return products;

	}
	public void addProduct(Product prod)
	{
		products.add(prod);
	}
	public Product getProductById(int prodid)
	{
		return products.stream()
				.filter(p-> p.getProdid()==prodid)
				.findFirst().get();
	}
}
```

HomeController.java (updated)

```
package com.simpleApp;

import java.util.List;

import org.springframework.beans.factory.annotation.Autowired;
```

```
import org.springframework.web.bind.annotation.PathVariable;

import org.springframework.web.bind.annotation.PostMapping;

import org.springframework.web.bind.annotation.RequestBody;

import org.springframework.web.bind.annotation.RequestMapping;

import org.springframework.web.bind.annotation.RestController;

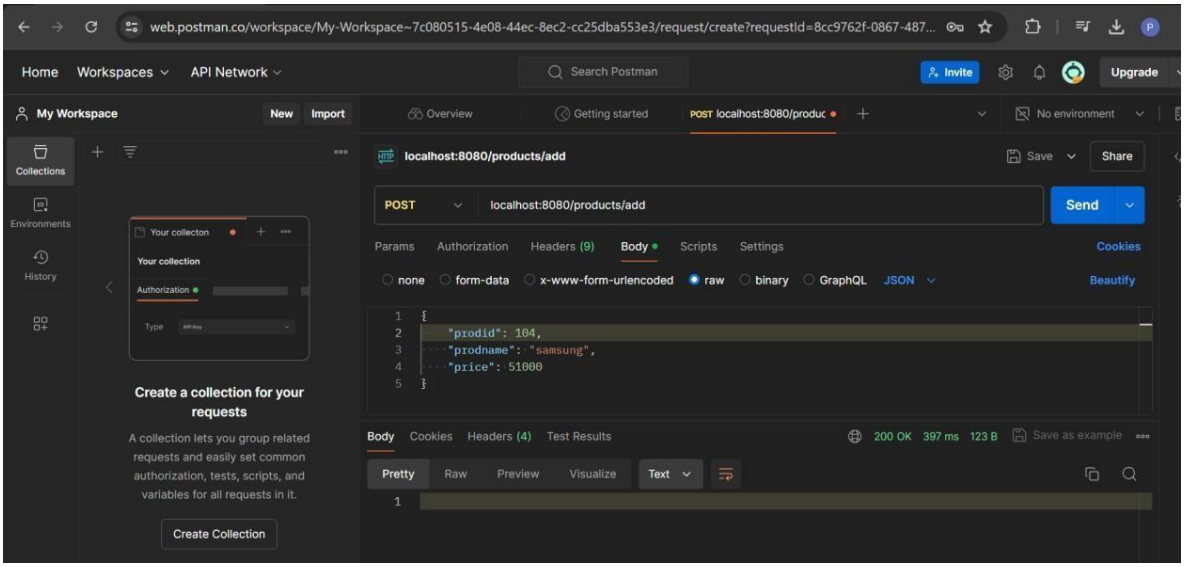
@RestController
public class HomeController
{
	@Autowired
	private ProductService service;

	@RequestMapping("/")
	public String greet()
	{
		return "welcome";
	}
	@RequestMapping("/about")
	public String aboutpage()
	{
		return "Iam about myself";
	}
	@RequestMapping("/products")
	public List<Product> getProducts()
	{
		return service.getProducts();
	}
	@RequestMapping("/products/{prodid}")
	public Product getProductById(@PathVariable int prodid)
	{
		return service.getProductById(prodid);
	}
	@PostMapping("/products/add")
	public void addProduct(@RequestBody Product prod)
	{
		service.addProduct(prod);
	}
}
```

In Postman: select POST, URL localhost:8080/products/add, Body -> raw -> JSON, e.g. {"prodid":104,"prodname":"samsung","price":51000}, click Send (Status 200 OK).
Verify by GET localhost:8080/products/ — the new item (prodid 104, samsung, 51000) now appears in the list.

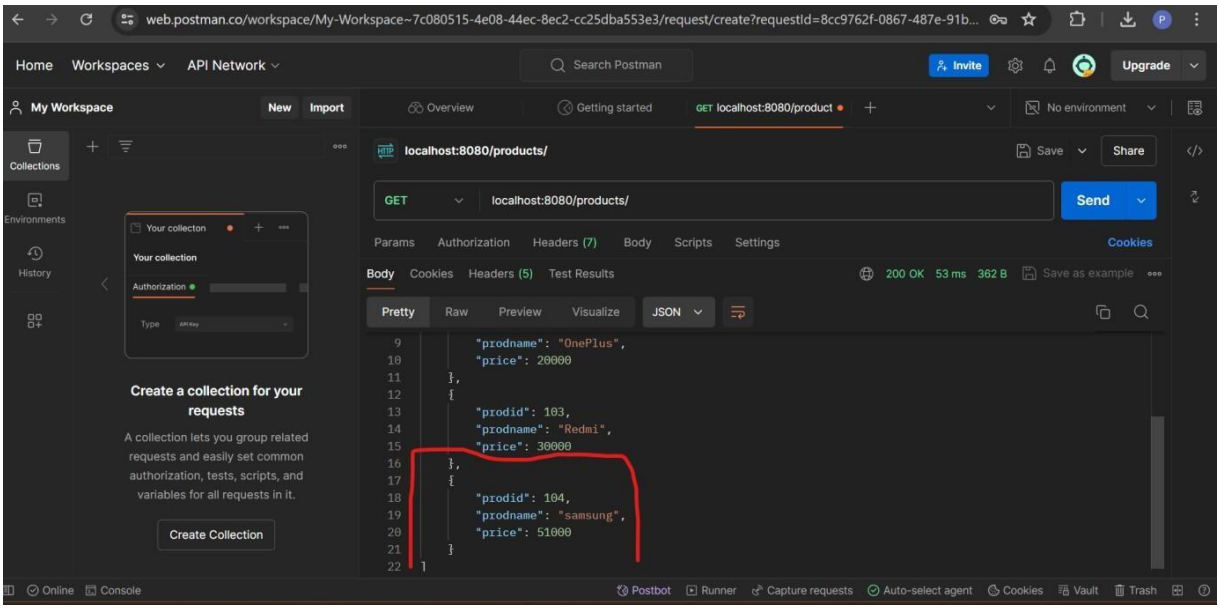
OUTPUT

Goto Postman Select POST-> Body-> JSON CLick Send



Display all the items and check whether it is updated.

Type GET localhost:8080/products/



RESULT

EX. NO:10 DATE:	UPDATING DATA IN DATABASE USING PUT API

AIM

To update data in database using using PUT api and to delete data from database using DELETE api.

PROCEDURE

PUT API

Change the type of method as PUT to update, go to Body->raw, type the data you wanted to change.
Example: ItemId 101 changed as "Cannon" — PUT localhost:8080/products/update with body {"prodid":101,"prodname":"cannon","price":50000} returns 200 OK.
Display the updated data using a GET request to localhost:8080/products/ — item 101 now shows "cannon".

DELETE API

Delete ItemId: 102 — change Request to DELETE and type localhost:8080/products/102.
Item 102 is deleted -> Display all items using a GET request and check. The list now shows only 101 (Iphone/cannon), 103 (Redmi) and any items added afterwards, confirming 102 has been removed.

SourceCode for UPDATE and DELETE

HomeController.java

```
package com.simpleApp;

import java.util.List;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.DeleteMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.PutMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class HomeController
{
	@Autowired
	private ProductService service;

	@RequestMapping("/")
	public String greet()
	{
		return "welcome";
	}
	@RequestMapping("/about")
	public String aboutpage()
```

```
	{
		return "Iam about myself";
	}
	@RequestMapping("/products")
	public List<Product> getProducts()
	{
		return service.getProducts();
	}
	@RequestMapping("/products/{prodid}")
	public Product getProductById(@PathVariable int prodid)
	{
		return service.getProductById(prodid);
	}
	@PostMapping("/products/add")
	public void addProduct(@RequestBody Product prod)
	{
		service.addProduct(prod);
	}
	@PutMapping("/products/update")
	public void updateProduct(@RequestBody Product prod)
	{
		service.updateProduct(prod);
	}
	@DeleteMapping("/products/{prodid}")
	public void deleteProduct(@PathVariable int prodid)
	{
		service.deleteProduct(prodid);
	}
}
```

ProductService.java

```
package com.simpleApp;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;

import org.springframework.stereotype.Service;

import com.simpleApp.Product;

@Service
public class ProductService
{
	private List<Product> products=new ArrayList<>(Arrays.asList(
				new Product(101,"Iphone",50000),
				new Product(102,"OnePlus",20000),
				new Product(103,"Redmi",30000)
```

```
                        )
                );

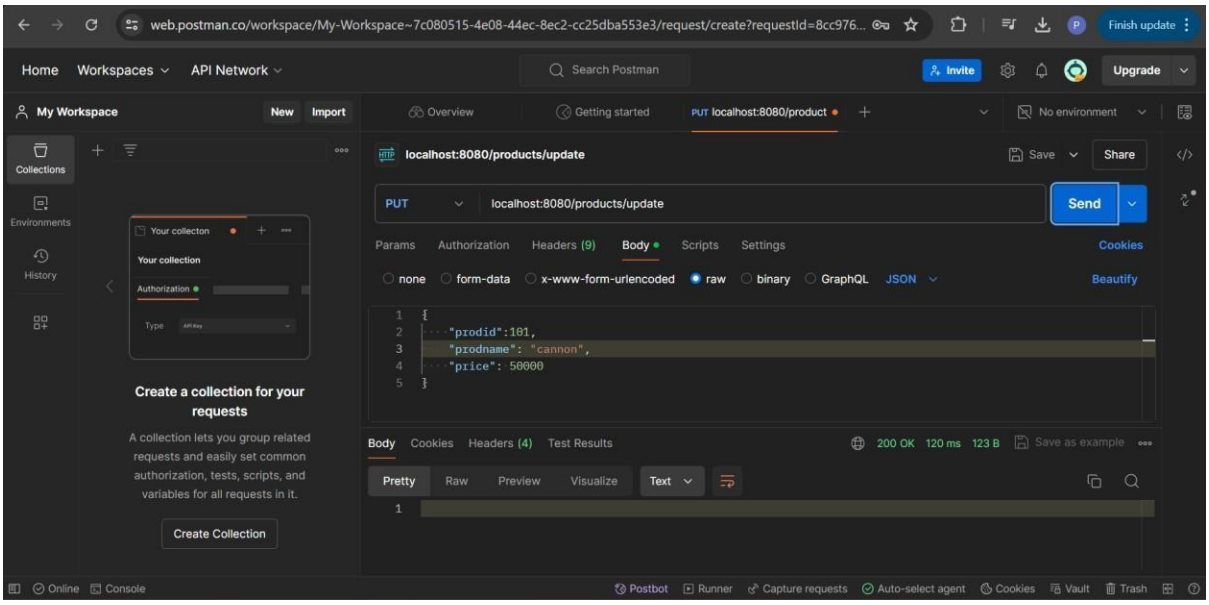
        public List<Product> getProducts()
        {
                return products;

        }
        public void addProduct(Product prod)
        {
                products.add(prod);
        }
        public Product getProductById(int prodid)
        {
                return products.stream()
                                .filter(p-> p.getProdid()==prodid)
                                .findFirst().get();
        }
        public void updateProduct(Product prod)
        {
                int index=0;
                for(int i=0;i<products.size();i++)
                        if(products.get(i).getProdid()==prod.getProdid())
                                index=i;
                products.set(index, prod);

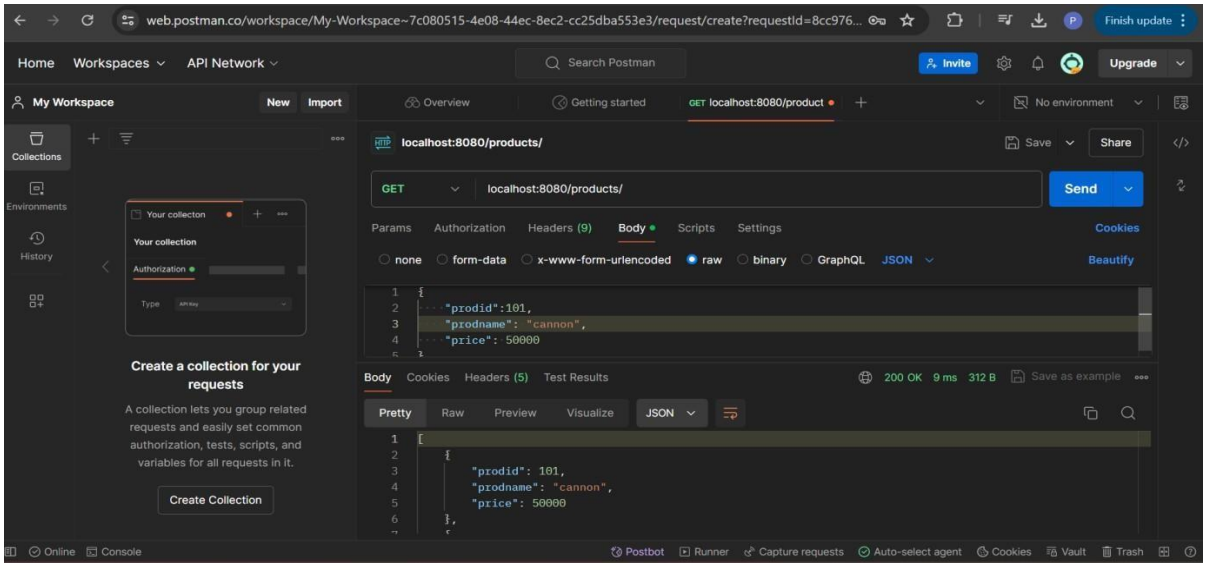
        }
        public void deleteProduct(int prodId)
        {
                int index=0;
                for(int i=0;i<products.size();i++)
                        if(products.get(i).getProdid()==prodId)
                                index=i;
                products.remove(index);
        }
}
```

OUTPUT

PUT API

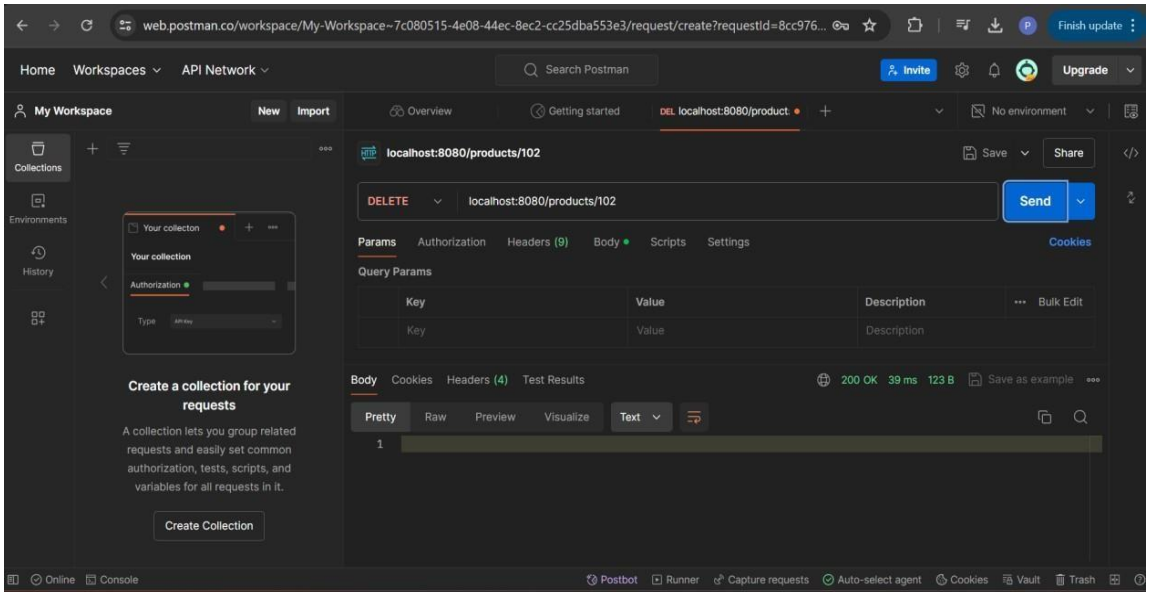


GET API

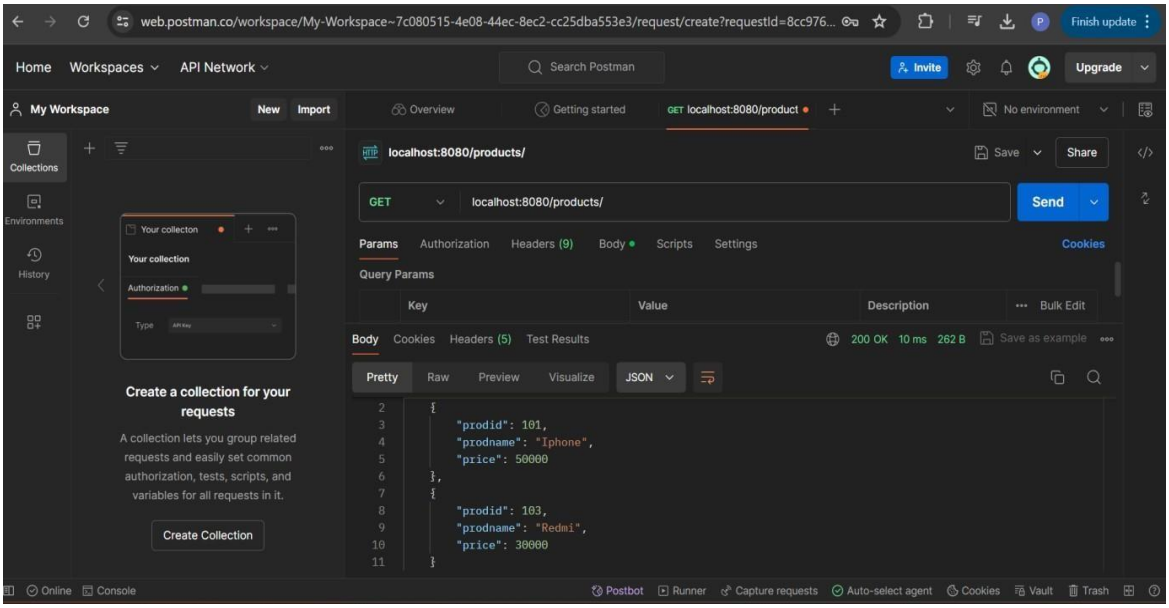


DELETE API

Change Request to **DELETE** and type as **localhost:8080/products/102**



ItemId102 **Deleted-> Display** all Items using **GET** request and check



RESULT

EX. NO:11 DATE:	RESPONSE CODE 200, 400, 500 WITH SPRING BOOT

AIM

To simulate the error code 200, 404, 500 using SPRING boot.

PROCEDURE

Error Code 404: This error code appears if the requested function is not implemented in the controller and service class.

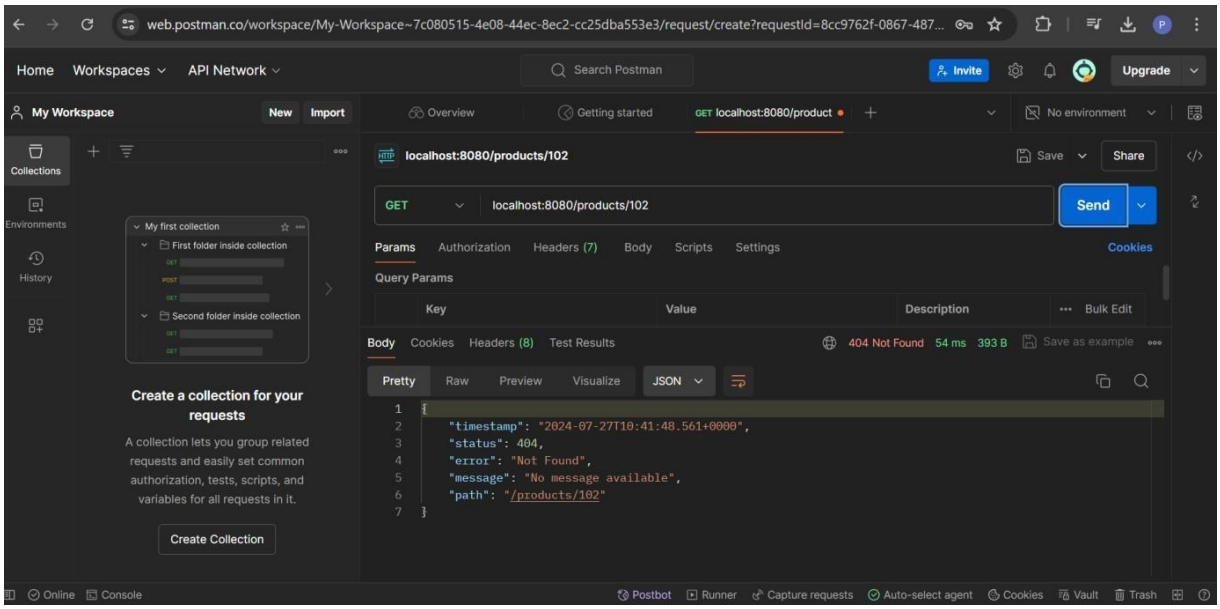
Example: GET localhost:8080/products/102 (after 102 has been deleted) returns HTTP 404 Not Found with a JSON error body:
{"timestamp":"...","status":404,"error":"Not Found","message":"No message available","path":"/products/102"}.

Error Code 500: This error appears if the requested item is not present in the data (e.g. an invalid product id such as 109 that never existed).

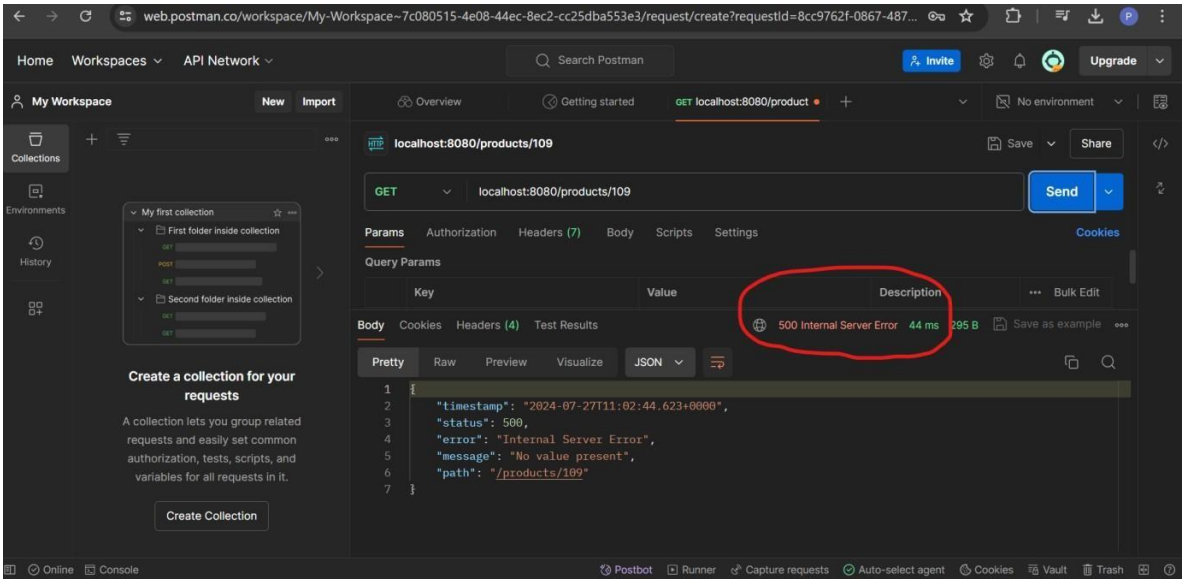
Example: GET localhost:8080/products/109 returns HTTP 500 Internal Server Error with a JSON error body:
{"timestamp":"...","status":500,"error":"Internal Server Error","message":"No value present","path":"/products/109"}.

OUTPUT

Error Code 404: This error code appears if the requested function is not implemented in the controller and service class.



Error Code 500 : if requested item not present in data



RESULT